

Maîtriser le projet

De zéro à la soutenance

Détection des véhicules éloignés, de petite taille et dans les embouteillages

Scène de trafic urbain appliquée au contexte béninois :
proposition d'amélioration basée sur YOLOv8

AMA, Cohorte 2, Projet Intégrateur 1, Groupe 5

Aïchatou TRAORE, Benoît DJOSSOU, Andréa AFOUDA

Comment lire ce document

Ce document est écrit pour une personne qui n'a jamais fait de deep learning. Chaque notion suit toujours le même chemin : une analogie du quotidien, une définition décodée mot à mot, un exemple chiffré avec les vrais nombres du projet, l'endroit exact où la notion vit dans le code, enfin le piège classique et la question de jury associée.

Règle de vérité. Toute affirmation technique est suivie de sa source sous la forme [fichier:lignes]. Quand une chose n'existe pas dans le dépôt, c'est écrit noir sur blanc. Trois étiquettes sont utilisées : [fait et vérifié] [fait mais non vérifiable dans le dépôt] [pas fait].

Si vous n'avez qu'une heure : lisez la section 0, puis la section 6, puis les fiches de survie de la section 9.

Table des matières

0	Le projet en une page	1
1	Problème et cadrage	3
1.1	Le contexte : pourquoi ce sujet, maintenant	3
1.2	La problématique, énoncée sans détour	3
1.3	Le périmètre : ce que nous faisons et ce que nous ne faisons pas	4
1.4	Notion : la détection d'objets	4
1.5	Notion : la boîte englobante	4
1.6	Notion : les classes	5
2	Les données	7
2.1	Le jeu source, et pourquoi celui-là	7
2.2	Notion : le format d'annotation YOLO	8
2.3	Notion : la conversion des classes	9
2.4	Les découpes train, validation, test	10
2.5	Notion : le déséquilibre des classes	11
2.6	Notion : la fuite de données	11
3	Le modèle	13
3.1	Notion : le réseau de neurones convolutif	13
3.2	YOLOv8 en trois blocs	13
3.3	Notion : le score de confiance	14
3.4	Notion : l'IoU, le recouvrement	15
3.5	Notion : la suppression des non-maxima	16
3.6	Notion : la fonction de perte	17
4	L'entraînement	18
4.1	Notion : le transfert d'apprentissage	18
4.2	Chaque hyperparamètre, expliqué	18
4.3	Notion : l'augmentation de données	20
4.4	Lire les courbes d'entraînement	21
4.5	Notion : le surapprentissage	22
5	L'évaluation	24
5.1	Notion : vrai positif, faux positif, faux négatif	24
5.2	Notion : précision, rappel, F1	25
5.3	Notion : la courbe précision-rappel	26
5.4	Notion : la mAP	26
5.5	Notion : la matrice de confusion	27
5.6	Notion : le débit, ou FPS	28
5.7	Notion : le rappel par taille d'objet	29
6	Nos résultats	31
6.1	Le cadre de la mesure	31
6.2	Ligne par ligne	31
6.3	Ce que « petit » veut dire concrètement	32
6.4	Qui sont les petits objets ? Une vérification supplémentaire	33
6.5	Pourquoi l'écart explose sur les petits objets	33
6.6	Ce que ces chiffres ne prouvent pas	34
6.7	Biais et limites, la liste complète	35

7	Le code de bout en bout	37
7.1	La carte du dépôt	37
7.2	Le flux général	37
7.3	Préalable : la ligne de commande sous Windows PowerShell	37
7.4	configs/import.yaml	38
7.5	configs/dataset.yaml	38
7.6	configs/entrainement.yaml	39
7.7	scripts/download_bmd45_subset.py	39
7.8	scripts/import_dataset.py	39
7.9	scripts/train_yolo.py	40
7.10	scripts/evaluate.py, le cœur méthodologique	41
7.11	scripts/compare_models.py, le livrable	42
7.12	scripts/detect_image.py	43
7.13	scripts/tracer_courbes.py	43
7.14	notebooks/colab_entrainement.ipynb	44
7.15	Le chemin complet, en six commandes	45
8	Les graphiques	46
8.1	La grille de 10 courbes d'Ultralytics	46
8.2	Notre figure d'article, deux panneaux	47
8.3	La courbe précision-rappel	48
8.4	La matrice de confusion normalisée	49
8.5	Les deux images de démonstration	50
9	Fiches de survie	52
9.1	Glossaire de tous les termes définis dans ce document	53
9.2	Script oral de 5 minutes	54
9.3	Les 20 questions de jury les plus dangereuses	56
9.4	Journal de vérité : incohérences relevées dans le dépôt	59
9.5	Le classement fait, non vérifiable, pas fait	61

0 Le projet en une page

À relire en dernier, juste avant d'entrer en salle.

L'idée en une phrase

Un détecteur d'objets générique, entraîné ailleurs, ne voit presque pas les véhicules lointains dans un embouteillage. Nous l'avons réentraîné sur du trafic urbain dense, et nous avons mesuré, sur les mêmes images de test, que sa capacité à retrouver les tout petits véhicules passe de 12 sur 100 à 69 sur 100. [results/comparaison.md:15]

Les cinq phrases qui résument tout

1. **Le contexte.** Le Bénin déploie un système national de vidéoprotection, décidé en Conseils des Ministres du 11 mars et du 3 juin 2026. La vidéo-verbalisation automatique commence par un maillon unique : détecter le véhicule. [article/sections/intro.tex:4-12]
2. **Le problème.** Les détecteurs pré-entraînés échouent sur les véhicules éloignés, petits à l'image, ou masqués dans les embouteillages, qui sont justement les configurations dominantes du trafic ouest-africain. [README.md:14-23]
3. **Ce que nous avons fait.** Un fine-tuning de YOLOv8n sur 3 000 images de trafic urbain dense (BMD-45, Bengaluru), puis une comparaison chiffrée avec le modèle d'origine sur exactement le même jeu de test. [article/sections/methode.tex:17-25]
4. **Le résultat.** Toutes les métriques progressent, et le gain est d'autant plus fort que l'objet est petit : +469,4 % de rappel sur les petits, +135,6 % sur les moyens, +20,2 % sur les grands. [results/comparaison.md:15-17]
5. **La limite assumée.** Il n'existe aucun jeu de données public béninois. Nous démontrons que l'adaptation de domaine fonctionne sur un trafic structurellement comparable, pas que le modèle est prêt pour Cotonou. [article/sections/discussion.tex:57-66]

Les neuf chiffres à connaître par cœur

Métrique	Pré-entraîné	Fine-tuné	Écart
mAP@0,5	0,4380	0,8252	+88,4 %
mAP@0,5:0,95	0,2959	0,6443	+117,8 %
Précision	0,4959	0,8327	+67,9 %
Rappel	0,4172	0,7138	+71,1 %
F1-score	0,4532	0,7687	+69,6 %
Débit (FPS)	37,2	43,2	+16,2 %
Rappel petits (804 objets)	0,1219	0,6940	+469,4 %
Rappel moyens (1 415 objets)	0,3731	0,8792	+135,6 %
Rappel grands (332 objets)	0,7922	0,9518	+20,2 %

[results/comparaison.md:9-17] **[fait et vérifié]**

Le protocole, en quatre garanties

1. **Mêmes images, mêmes boîtes.** Les deux modèles sont évalués sur les mêmes 300 images de test et les mêmes 2 551 boîtes de référence. [article/sections/methode.tex:137-140]
2. **Mêmes classes.** Les 14 catégories de BMD-45 sont ramenées à 4 classes alignées sur COCO, et les étiquettes sont réindexées à la volée vers l'espace du modèle évalué. [scripts/evaluate.py:40, 73-108]

3. **Jeu de test isolé.** Le test est prélevé sur la validation de façon déterministe (graine 42) et n'intervient ni dans l'entraînement ni dans la sélection du modèle. [scripts/import_dataset.py:166-190]
4. **Aires normalisées.** Les seuils de taille COCO sont appliqués après remise à l'échelle des aires à 640 pixels, sinon tous les véhicules seraient classés « grands » et la mesure ne mesurerait rien. [scripts/evaluate.py:53-61, 213]

La phrase de conclusion à prononcer

🗨 Devant le jury

« Nous ne prétendons pas avoir adapté YOLOv8 au trafic béninois : aucun jeu de données béninois public n'existe. Nous démontrons, sur un trafic dense structurellement comparable, qu'un fine-tuning modeste lève précisément le verrou visé, celui des véhicules petits et éloignés, et nous livrons le protocole d'évaluation clef en main pour le refaire le jour où un jeu béninois existera. » [article/sections/conclusion.tex:15-25]

1 Problème et cadrage

1.1 Le contexte : pourquoi ce sujet, maintenant

La République du Bénin a engagé la mise en place d'un système national de vidéoprotection couvrant cinq villes et les postes frontaliers, décidé en Conseils des Ministres du 11 mars et du 3 juin 2026. [article/sections/intro.tex:4-8]

Un réseau de caméras ouvre la voie à la **vidéo-verbalisation** : constater une infraction routière automatiquement, à partir des images. C'est une chaîne de traitements, et chaque maillon dépend du précédent.

Rang	Maillon	Ce qu'il produit
1	Détection des véhicules	une boîte autour de chaque véhicule
2	Suivi multi-objets	un identifiant qui suit le même véhicule d'image en image
3	Détection de la plaque	une boîte autour de la plaque
4	Reconnaissance de caractères	le texte de la plaque
5	Règles d'infraction	« ce véhicule a grillé le feu »
6	Restitution	un tableau de bord, un procès-verbal

[article/sections/intro.tex:8-12]



L'idée en une phrase

Si le maillon 1 rate un véhicule, aucun maillon suivant ne peut le rattraper : un véhicule non détecté est un véhicule qui n'existe pas pour le système. C'est pour cela que le projet se concentre sur la détection seule. [article/sections/intro.tex:15-17]

1.2 La problématique, énoncée sans détour

Trois phénomènes du trafic urbain béninois mettent en difficulté les détecteurs génériques [README.md:16-21] :

- les véhicules éloignés occupent **peu de pixels** à l'image ;
- plusieurs véhicules sont regroupés ou **se chevauchent** ;
- les motos-taxis (zémidjans) circulent de manière **moins prévisible**.

Un chiffre publié situe l'enjeu : des détecteurs entraînés sur UA-DETRAC, un benchmark occidental classique, n'atteignent que 33,6 % de mAP@0,5:0,95 sur du trafic urbain dense de pays émergent, contre 83,8 % pour des modèles entraînés sur des données du même domaine, soit un facteur 2,5. [docs/DATASETS.md:26-30 ; article/sections/intro.tex:21-26]



Vérification dans le dépôt

Ce facteur 2,5 ne vient pas de nos mesures : c'est un résultat publié par les auteurs du jeu de données BMD-45, que nous citons. **[fait mais non vérifiable dans le dépôt]** au sens où nous ne l'avons pas reproduit nous-mêmes. Notre propre facteur, mesuré sur notre jeu de test, est 2,2 sur la mAP@0,5:0,95 (de 0,2959 à 0,6443). [article/sections/discussion.tex:22-33]

1.3 Le périmètre : ce que nous faisons et ce que nous ne faisons pas

Dans le périmètre	Hors périmètre
Détection de véhicules (voiture, moto, bus, camion)	Lecture de plaques (ANPR, OCR)
Fine-tuning de YOLOv8	Suivi multi-objets et identifiants persistants
Évaluation chiffrée et comparaison	Règles d'infraction, estimation de vitesse
Analyse par taille d'objet	Procès-verbaux, interface web

[README.md:45-50] **[fait et vérifié]**

Devant le jury

Question probable : « Votre projet s'appelle vidéoprotection, mais vous ne lisez même pas les plaques. Pourquoi ? »

Réponse en trois phrases : le pipeline complet compte six maillons et tous dépendent du premier. Nous avons choisi de traiter le premier maillon correctement plutôt que les six superficiellement. La lecture de plaques est explicitement hors périmètre et documentée comme telle depuis le début du projet.

1.4 Notion : la détection d'objets

 **Analogie.** Imaginez qu'on vous montre la photo d'un embouteillage et qu'on vous demande de tracer un rectangle au feutre autour de chaque véhicule, puis d'écrire à côté de chaque rectangle « voiture », « moto », « bus » ou « camion ». C'est exactement le travail d'un détecteur d'objets. Ce n'est pas la même tâche que « dis-moi s'il y a une voiture sur cette photo » (cela s'appelle la **classification**), ni que « colorie exactement les pixels de la voiture » (cela s'appelle la **segmentation**).

 **Définition, mot à mot.** La détection d'objets consiste à produire, pour une image donnée, une liste de triplets : une **boîte englobante** (les coordonnées d'un rectangle), une **classe** (l'étiquette de l'objet) et un **score de confiance** (un nombre entre 0 et 1 qui dit à quel point le modèle y croit).

 **En français courant.** « Où, quoi, et à quel point j'en suis sûr. »

 **Exemple chiffré.** Sur l'image de démonstration du projet, le modèle pré-entraîné produit 4 boîtes, le modèle fine-tuné en produit 44, sur la même photo et avec le même seuil de confiance de 0,25. [article/figures/LISEZMOI.md:5-8]

 **Chez nous.** La détection est appelée par la fonction `modele.predict(...)` dans deux fichiers : `scripts/detect_image.py:62` pour les visuels, et `scripts/evaluate.py:197-203` pour la mesure. Si vous changez `conf=0.25` en `conf=0.05`, le modèle produira beaucoup plus de boîtes : le rappel montera un peu et la précision chutera. [scripts/detect_image.py:42 ; scripts/evaluate.py:43]

 **Piège et question de jury.** Le débutant croit qu'un détecteur « reconnaît » les objets comme un humain. Il calcule en réalité un score pour un très grand nombre de positions possibles, puis ne garde que les meilleures. **Question de jury :** « que se passe-t-il si deux véhicules se superposent presque exactement ? » Réponse : la suppression des non-maxima (section 3) risque d'en supprimer un, et c'est une des causes de nos faux négatifs en embouteillage.

1.5 Notion : la boîte englobante

 **Analogie.** C'est le cadre rectangulaire que dessine un appareil photo autour d'un visage. Il ne suit pas les contours, il encadre.

 **Définition, mot à mot.** Une boîte englobante (*bounding box* en anglais) est le plus petit rectangle à côtés horizontaux et verticaux qui contient entièrement l'objet. Dans le format YOLO

utilisé par le projet, elle s'écrit avec quatre nombres **normalisés**, c'est-à-dire ramenés entre 0 et 1 en divisant par la largeur ou la hauteur de l'image :

$$(c_x, c_y, w, h)$$

où c_x et c_y sont les coordonnées du **centre** de la boîte, w sa largeur et h sa hauteur.

☞ **En français courant.** « Le centre du rectangle est à tel pourcentage de la largeur et tel pourcentage de la hauteur de l'image, et il mesure tel pourcentage de large sur tel pourcentage de haut. »

📊 **Exemple chiffré.** Le script de téléchargement convertit les boîtes du jeu source, qui sont données en pixels sous la forme (coin supérieur gauche, largeur, hauteur), vers ce format :

```
98 def yolo_line(category_id: int, bbox_xywh: list[float], width: int, height: int) -> str:
99     x, y, w, h = bbox_xywh
100     cx = (x + w / 2.0) / width # centre horizontal, ramené entre 0 et 1
101     cy = (y + h / 2.0) / height # centre vertical
102     nw = w / width # largeur relative
103     nh = h / height # hauteur relative
```

[scripts/download_bmd45_subset.py:98-111]

Concrètement, une voiture dont le coin supérieur gauche est à $x = 800$, $y = 450$, large de 120 pixels et haute de 90, dans une image de 1920 par 1080, donne : $c_x = (800 + 60)/1920 = 0,448$; $c_y = (450 + 45)/1080 = 0,458$; $w = 120/1920 = 0,0625$; $h = 90/1080 = 0,0833$. La ligne écrite dans le fichier d'étiquettes sera donc, pour une voiture (classe 0) : 0 0.448000 0.458333 0.062500 0.083333.

📁 **Chez nous.** Le chemin inverse est fait à l'évaluation : la fonction `charger_verite_terrain` relit ces nombres normalisés et les retransforme en pixels absolus, sous la forme (coin haut gauche, coin bas droit), qui est le format nécessaire au calcul de recouvrement. [scripts/evaluate.py:161-174]

⚠ **Piège et question de jury.** Confondre « coin supérieur gauche plus dimensions » et « centre plus dimensions » est l'erreur la plus fréquente, et elle décale toutes les boîtes d'une demi-largeur. Le symptôme est spectaculaire : les métriques s'effondrent sans raison apparente. **Question de jury :** « pourquoi normaliser les coordonnées ? » Réponse : pour que l'étiquette reste valable quelle que soit la résolution de l'image, y compris après le redimensionnement à 640 pixels appliqué à l'entrée du réseau.

1.6 Notion : les classes

🌱 **Analogie.** Une classe, c'est le tiroir dans lequel on range un objet. Le projet n'a que quatre tiroirs, et il faut décider dans lequel va chaque type de véhicule du jeu de données d'origine, qui en compte quatorze.

🔑 **Définition, mot à mot.** Une classe est une catégorie d'objet que le modèle sait nommer. Elle est identifiée par un **indice**, c'est-à-dire un numéro entier, et non par son nom : dans les fichiers d'étiquettes, on lit 0 et non *voiture*. La correspondance entre indice et nom est donnée par un fichier de configuration.

📊 **Exemple chiffré.** Le projet retient quatre classes, volontairement choisies parce qu'elles existent aussi dans COCO, le jeu de données sur lequel le modèle de référence a été entraîné :

Classe locale	Indice local	Classe COCO	Indice COCO
voiture	0	car	2
moto	1	motorcycle	3
bus	2	bus	5
camion	3	truck	7

[README.md:166-171 ; configs/dataset.yaml:24-28 ; scripts/evaluate.py:40]

📁 **Chez nous.** Cette table vit à trois endroits, et c'est voulu : la liste **CLASSES** dans `scripts/import_dataset.py:38`, le dictionnaire **LOCAL_VERS_COCO** dans `scripts/evaluate.py:40`, et le fichier `configs/dataset.yaml:24-28` qui est régénéré automatiquement. Si vous ajoutiez une cinquième classe sans équivalent COCO, la comparaison avec le modèle pré-entraîné deviendrait injuste : on lui demanderait de détecter une catégorie qu'il n'a jamais apprise. [article/sections/methode.tex:32-36]

⚠ **Piège et question de jury.** Croire que « bus » a le même numéro partout. Il vaut 2 chez nous et 5 dans COCO. Un oubli de conversion ne provoque aucune erreur visible : le programme tourne, et les métriques du modèle pré-entraîné sont simplement fausses. **Question de jury** : « comment savez-vous que vos deux modèles sont comparables ? » Réponse : parce que les étiquettes du jeu de test sont réindexées à la volée vers l'espace de classes du modèle évalué, et que le modèle COCO est en plus restreint aux quatre indices véhicules pour ne pas être pénalisé par des piétons ou des feux tricolores comptés en faux positifs. [scripts/evaluate.py:73-108, 123]

✖ À retenir

Trois mots à ne pas confondre : **classification** (quoi, sans où), **détection** (quoi et où, avec un rectangle), **segmentation** (quoi et où, au pixel près). Notre projet fait de la détection, sur quatre classes, avec des boîtes au format YOLO normalisé.

2 Les données

2.1 Le jeu source, et pourquoi celui-là

2.1.1 Le constat de départ

Il n'existe aucun jeu de données public de vision par ordinateur spécifique au Bénin pour la détection de véhicules. Les recherches ne remontent rien, ni sur Roboflow Universe, ni sur Hugging Face, ni dans la littérature. [docs/DATASETS.md:13-15]

C'est une contrainte à énoncer, pas à contourner. Elle a deux conséquences directes sur ce que le projet peut affirmer [docs/DATASETS.md:19-24] :

- nous ne pouvons pas mesurer l'écart de domaine *béninois*, faute de données béninoises pour le mesurer ;
- nous pouvons mesurer le gain apporté par un fine-tuning sur du *trafic urbain dense*, en particulier sur les véhicules de petite taille. C'est l'objet du projet.

2.1.2 BMD-45 : la source retenue

Caractéristique	Valeur
Identifiant Hugging Face	iisc-aim/BMD-45
Volume total	45 000 images, 480 000 boîtes annotées
Origine	plus de 3 600 caméras CCTV opérationnelles, Bengaluru (Inde)
Catégories	14 catégories fines de véhicules
Publication	CVPR 2026 (Findings)
Licence	CC BY 4.0

[article/sections/methode.tex:5-15 ; scripts/download_bmd45_subset.py:27, 65-81]

Trois raisons de l'avoir choisi, et elles répondent une par une aux critères de sélection du projet [article/sections/methode.tex:11-15 ; docs/PROCEDURE.md:152-157] :

1. **Caméras fixes de vidéoprotection**, et non caméras embarquées dans un véhicule. C'est le cas d'usage visé.
2. **Trafic urbain dense**, donc beaucoup d'occlusions et beaucoup de petits objets au fond de la scène.
3. **Forte proportion de deux-roues**, trait structurel partagé avec le trafic béninois dominé par les zémidjans.

⚠ Vérification dans le dépôt

Le rapprochement Inde et Bénin est un argument de *similarité structurelle*, pas une équivalence. Le dépôt l'écrit lui-même : « nos résultats établissent que l'adaptation à un domaine de trafic dense de ce type est possible et fortement bénéfique ; ils ne démontrent pas que le modèle obtenu est adapté au trafic béninois lui-même ». [article/sections/discussion.tex:62-66]

2.1.3 L'extraction du sous-ensemble

45 000 images sont hors budget de calcul. Le script `download_bmd45_subset.py` lit le jeu **en flux** (*streaming*), c'est-à-dire sans le télécharger entièrement, et n'écrit sur le disque que le nombre d'images demandé. [scripts/download_bmd45_subset.py:1-15]

```
1| python scripts/download_bmd45_subset.py --train 2400 --val 600 --seed 42
```

[README.md:116 ; valeurs par défaut : scripts/download_bmd45_subset.py:33-35]

Le mélange aléatoire du flux utilise la graine 42 pour l'entraînement et la graine 43 pour la validation [scripts/download_bmd45_subset.py:236, 247]. La graine (*seed*) est le nombre qui initialise le générateur aléatoire : avec la même graine, le même tirage se reproduit à l'identique. C'est ce qui rend le sous-ensemble reproductible par n'importe quel membre du groupe.

✦ À retenir

Trois détails de ce script sont défendables devant un jury. **Un** : les images sont réenregistrées en JPEG avec une qualité de 95 et non la valeur par défaut d'environ 75, parce qu'une compression agressive dégraderait visiblement les petits objets, qui sont le cœur du sujet. [scripts/download_bmd45_subset.py:179-182] **Deux** : le script reprend là où il s'était arrêté si on l'interrompt, et supprime une éventuelle paire image/étiquette incomplète pour repartir d'un état propre. [scripts/download_bmd45_subset.py:114-133] **Trois** : les coordonnées sont bornées entre 0 et 1 par précaution, au cas où une boîte dépasserait de un ou deux pixels. [scripts/download_bmd45_subset.py:105-109]

2.2 Notion : le format d'annotation YOLO



Analogie. C'est une fiche cartonnée par photo. Sur la fiche, une ligne par véhicule, et sur chaque ligne cinq nombres : le tiroir, puis les quatre nombres du rectangle. Rien d'autre, pas de nom, pas de commentaire.



Définition, mot à mot. Le format YOLO impose une arborescence stricte : un dossier `images/` et un dossier `labels/` côte à côte. À chaque image `photo.jpg` correspond un fichier `photo.txt` de même nom. Chaque ligne du fichier texte contient :

`indice_de_classe cx cy w h`

séparés par des espaces, les quatre derniers nombres étant normalisés entre 0 et 1.



Exemple chiffré. Un fichier d'étiquettes de notre jeu ressemble à ceci (trois véhicules : deux motos et une voiture) :

```
1 | 1 0.512340 0.734500 0.041200 0.062700
2 | 1 0.548900 0.741300 0.038700 0.059100
3 | 0 0.221000 0.612000 0.104000 0.088000
```

Une image sans aucun véhicule a un fichier texte vide, et non pas de fichier du tout : le script d'import écrit toujours le fichier, même vide. [scripts/import_dataset.py:155-157]



Chez nous. L'arborescence finale, produite par l'import, est la suivante :

```
1 | data/dataset/
2 |   train/images/ train/labels/
3 |   val/images/ val/labels/
4 |   test/images/ test/labels/
```

[scripts/import_dataset.py:133-139 ; configs/dataset.yaml:19-22]



Piège et question de jury. Un décalage entre le nom de l'image et le nom du fichier d'étiquettes rend l'image *silencieusement* sans annotation : aucune erreur, mais tous ses véhicules deviennent des faux négatifs. **Question de jury** : « comment gérez-vous les images sans véhicule ? » Réponse : elles restent dans le jeu avec un fichier d'étiquettes vide, ce qui apprend au modèle à ne rien détecter sur du bitume nu et réduit les faux positifs.

2.3 Notion : la conversion des classes

 **Analogie.** Vous recevez un carton d'archives étiqueté en quatorze catégories très fines (berline, citadine, monospace, camionnette, etc.) et vous devez le reclasser en quatre tiroirs seulement. Certaines catégories n'ont aucun tiroir possible : vous les mettez de côté explicitement, vous ne les jetez pas « par oubli ».

 **Définition, mot à mot.** Le **remappage** est la table qui associe chaque classe du jeu source à une classe du projet, ou à la mention **ignorer**. Il est écrit à la main dans `configs/import.yaml`, et appliqué par `scripts/import_dataset.py`.

 **Exemple chiffré.** Voici la table réelle, telle qu'elle figure dans le dépôt :

Classes BMD-45	Classe retenue	Indice COCO
Hatchback, Sedan, SUV, MUV, Van	voiture	2 (car)
Two-wheeler	moto	3 (motorcycle)
Bus, Mini-bus, Tempo-traveller	bus	5 (bus)
Truck, LCV	camion	7 (truck)
Three-wheeler, Bicycle, Other	ignorées, aucun équivalent COCO	

[`configs/import.yaml:16-52` ; `article/sections/methode.tex:44-56`] **[fait et vérifié]**

Le compte est bon : $5 + 1 + 3 + 2 + 3 = 14$ catégories, toutes traitées.

 **Chez nous.** La conversion se fait ligne par ligne dans le fichier d'étiquettes. Le code est court et mérite d'être lu :

```

119 def convertir_label(chemin: Path, table: dict[int, int]) -> list[str]:
120     lignes = []
121     for ligne in chemin.read_text(encoding="utf-8").splitlines():
122         champs = ligne.split()
123         if len(champs) < 5: # ligne mal formee : on l'ignore
124             continue
125         source = int(float(champs[0]))
126         if source not in table: # classe mise a `ignorer` : on l'enleve
127             continue
128         champs[0] = str(table[source]) # on remplace l'indice source par le notre
129         lignes.append(" ".join(champs))
130     return lignes

```

[`scripts/import_dataset.py:119-130`]

Trois choses à remarquer. **Un** : seule la première colonne est modifiée, les coordonnées sont recopiées telles quelles, car elles sont déjà normalisées et indépendantes de la classe. **Deux** : une classe mise à **ignorer** n'est pas convertie en une classe « autre », elle est *effacée* de l'annotation. **Trois** : l'image, elle, est conservée. Un trois-roues reste donc visible sur l'image mais n'est plus annoté : le modèle apprend à ne pas le signaler.

 **Piège et question de jury.** Oublier une classe source. Le script refuse alors de continuer :

```

106     if inconnues:
107         print(f"\n{len(inconnues)} classe(s) absente(s) de configs/import.yaml : ...")
108         print("Ajoutez-les sous `correspondance`, en cible ou en `ignorer`.")
109         sys.exit("Import interrompu : la correspondance est incomplete.")

```

[`scripts/import_dataset.py:106-112`]

C'est un choix de conception fort : mieux vaut un arrêt bruyant qu'un résultat faux. **Question de jury** : « pourquoi ignorer les trois-roues, n'est-ce pas se faciliter la tâche ? » Réponse : les conserver reviendrait à demander au modèle pré-entraîné de détecter une catégorie que COCO ne contient pas, donc à truquer la comparaison *en sa défaveur*. L'exclusion protège le modèle de référence, pas le nôtre.

2.4 Les découpes train, validation, test

2.4.1 Notion : à quoi servent trois jeux séparés

 **Analogie.** L'entraînement, c'est le cahier d'exercices avec les corrigés. La validation, c'est le devoir blanc que le professeur utilise pour ajuster son cours. Le test, c'est l'examen final : on ne le regarde qu'une fois, à la fin. Réviser sur le sujet de l'examen, c'est de la triche, même involontaire.

 **Définition, mot à mot.** Le jeu d'**entraînement** sert à ajuster les paramètres du modèle. Le jeu de **validation** sert à surveiller l'entraînement et à choisir quel instantané du modèle garder. Le jeu de **test** sert uniquement à produire le chiffre final, et n'intervient à aucun autre moment.

 **Exemple chiffré.** Notre découpe est produite en deux temps. Le script de téléchargement crée deux dossiers, `train` (2 400 images) et `val` (600 images), et écrit explicitement qu'il n'y a pas de jeu de test [scripts/download_bmd45_subset.py:252, 258]. Le script d'import constate cette absence et prélève la moitié de la validation pour en faire le test, avec la graine 42 :

```
179 images = sorted((val / "images").iterdir())
180 random.Random(graine).shuffle(images) # melange reproductible
181 for image in images[: len(images) // 2]: # la premiere moitie part en test
182     shutil.move(str(image), test / "images" / image.name)
```

[scripts/import_dataset.py:166-190]

Résultat : 2 400 images d'entraînement, 300 de validation, 300 de test.

[article/sections/methode.tex:21-25]

 **Chez nous.** Le commentaire du code explique le compromis : « sans jeu de test isolé, il n'y a pas de résultat mesurable, seulement une démonstration. Mieux vaut une validation plus petite qu'aucun test. » [scripts/import_dataset.py:168-170]

 **Piège et question de jury.** Utiliser le jeu de validation comme jeu de test. Les chiffres seraient optimistes, puisque c'est sur la validation que le meilleur instantané du modèle a été choisi. **Question de jury** : « votre jeu de test a-t-il servi à choisir quoi que ce soit ? » Réponse : non, il est dérivé avant l'entraînement et n'intervient ni dans la boucle d'entraînement ni dans la sélection du modèle.

[article/sections/methode.tex:170-173]

2.4.2 La volumétrie réelle

Split	Images	voiture	moto	bus	camion	Total
train	2 400	5 744	11 641	1 298	1 873	20 556
val	300	752	1 476	138	215	2 581
test	300	742	1 380	189	240	2 551
Total	3 000	7 238	14 497	1 625	2 328	25 688

[article/sections/methode.tex:71-82]

Vérification dans le dépôt

[fait et vérifié] Ce tableau a été recompté directement sur les fichiers d'étiquettes présents dans `data/dataset/` au moment de la rédaction de ce document : les 21 nombres correspondent exactement. C'est la vérification la plus facile à refaire devant un jury sceptique, et elle prend dix secondes.

Deux faits se déduisent de ce tableau, et ils sont utiles à l'oral :

- **Densité** : $25\,688 / 3\,000 = 8,56$ véhicules annotés par image en moyenne. C'est la signature d'un trafic dense, propice aux occlusions. [article/sections/methode.tex:60-64]
- **Dominance des deux-roues** : $14\,497 / 25\,688 = 56,4\%$ des instances sont des motos. C'est le point qui rapproche structurellement ce jeu du trafic béninois.

2.5 Notion : le déséquilibre des classes

 **Analogie.** Si vous apprenez une langue avec un manuel où 56 pages sur 100 parlent de motos et 6 pages seulement de bus, vous parlerez très bien des motos et vous hésiterez sur les bus. Le modèle fait exactement pareil.

 **Définition, mot à mot.** Un jeu est **déséquilibré** quand les effectifs par classe sont très différents. Le modèle voit beaucoup plus d'exemples de la classe majoritaire, apprend mieux à la reconnaître, et les métriques des classes minoritaires deviennent à la fois moins bonnes et moins stables, c'est-à-dire qu'elles varient beaucoup d'un entraînement à l'autre.

 **Exemple chiffré.** Sur le jeu d'entraînement : moto 11 641 instances (56,6 %), voiture 5 744 (27,9 %), camion 1 873 (9,1 %), bus 1 298 (6,3 %). Le rapport entre la classe la plus fréquente et la moins fréquente est de 9 pour 1. Sur le jeu de test, les bus ne comptent que 189 instances et les camions 240.

L'effet se lit directement dans les résultats du projet. Sur la courbe précision-rappel du run d'entraînement, l'aire sous la courbe vaut 0,910 pour la voiture, 0,868 pour la moto, 0,833 pour le bus et 0,760 pour le camion. [\[results/entraînements/yolov8n_benin/BoxPR_curve.png\]](#) [\[fait et vérifié\]](#)

 **Chez nous.** Le dépôt en tire une conséquence méthodologique explicite : aucune métrique par classe n'est rapportée dans l'article, précisément parce que les effectifs des classes minoritaires sont trop faibles pour être stables. [\[article/sections/discussion.tex:82-87\]](#) Le script d'import affiche d'ailleurs le tableau des effectifs et avertit : « une classe à moins de quelques dizaines d'instances en test ne donnera pas de métrique fiable ». [\[scripts/import_dataset.py:284-285\]](#)

 **Piège et question de jury.** Publier une métrique par classe calculée sur quelques dizaines d'objets, puis se faire démonter dessus. **Question de jury** : « quelle classe marche le moins bien et pourquoi ? » Réponse : le camion, avec 0,760 d'aire sous la courbe contre 0,910 pour la voiture. C'est la classe la moins représentée après le bus, et elle est visuellement confondable avec le bus, comme le montre la matrice de confusion (section 8).

2.6 Notion : la fuite de données

 **Analogie.** Le professeur donne un devoir blanc, puis, sans y penser, remet trois exercices identiques à l'examen. Les notes montent, mais elles ne mesurent plus rien.

 **Définition, mot à mot.** On parle de **fuite de données** (*data leakage*) quand une information du jeu de test se retrouve, directement ou indirectement, dans le jeu d'entraînement. Le cas le plus courant dans les jeux d'images publics est la présence d'images quasi identiques de part et d'autre de la séparation.

 **Exemple chiffré.** Notre risque est identifié et documenté : BMD-45 est constitué d'images de caméras **fixes**. Deux images prises par la même caméra à quelques secondes d'intervalle se ressemblent énormément. Si l'une part en entraînement et l'autre en test, les métriques absolues des deux modèles sont flattées. [\[article/sections/discussion.tex:89-95\]](#)

 **Chez nous.** Deux protections existent, et une limite subsiste. **Protection 1** : nos jeux d'entraînement et de validation viennent de deux splits différents du jeu source (les splits **train** et **val** de Hugging Face), et non d'un mélange que nous aurions fait nous-mêmes. [\[scripts/download_bmd45_subset.py:229-250\]](#) **Protection 2** : le test est dérivé de la validation, donc il n'a aucune image commune avec l'entraînement. **Limite** : rien ne garantit qu'une même caméra n'apparaisse pas dans les deux splits d'origine. Le dépôt l'écrit franchement plutôt que de le cacher.

Devant le jury

Question probable, et elle est piégeuse : « vos 0,82 de mAP ne sont-ils pas gonflés par une contamination entre vos splits ? »

Réponse en trois phrases : c'est un risque réel que nous documentons nous-mêmes, car BMD-45 est filmé par des caméras fixes et des images voisines peuvent se ressembler. Cette contamination, si elle existe, flatte les *deux* modèles de la même façon, donc elle n'invalide pas la comparaison, qui est notre résultat. Nos chiffres absolus doivent en revanche être lus comme des ordres de grandeur, ce que nous écrivons dans la section Limites.

 À retenir

La section 2 en une ligne : 3 000 images de trafic dense indien, 25 688 véhicules, 14 catégories ramenées à 4 alignées sur COCO, découpe 2 400 / 300 / 300 reproductible à la graine 42, dominée par les motos à 56 %, avec un risque de contamination assumé et déclaré.

3 Le modèle

⚠ Vérification dans le dépôt

Avertissement de méthode pour toute cette section. Le dépôt contient les *poids* du modèle et ses *réglages*, pas son schéma d'architecture. Les descriptions d'architecture ci-dessous relèvent de la connaissance générale sur YOLOv8 et sont signalées comme telles. Tout ce qui est chiffré et sourcé par un [fichier:ligne] vient bien du dépôt. Les deux comptes de paramètres donnés plus bas ont été obtenus en chargeant réellement les deux fichiers de poids du dépôt.

3.1 Notion : le réseau de neurones convolutif

🌱 **Analogie.** Imaginez une chaîne d'inspecteurs qui se passent une photo. Le premier ne regarde que de tout petits carrés de 3 pixels sur 3 et note seulement « ici il y a un bord clair vers foncé ». Le deuxième ne regarde plus la photo mais les notes du premier, et repère des assemblages de bords : un coin, un arc. Le troisième repère des roues, le quatrième des voitures. Aucun inspecteur ne sait ce qu'est une voiture ; c'est l'empilement qui le sait.

🔧 **Définition, mot à mot.** Un **réseau convolutif** est un empilement de couches. Une couche de **convolution** fait glisser un petit tableau de nombres, appelé **filtre** ou *noyau*, sur toute l'image, et calcule à chaque position une somme pondérée des pixels. Le résultat est une nouvelle image appelée **carte d'activation**, qui est claire là où le motif cherché est présent. Les nombres du filtre ne sont pas choisis par un humain : ce sont les **paramètres**, et l'entraînement consiste précisément à les régler.

🗨 **En français courant.** « On apprend automatiquement quels petits motifs chercher, puis quels assemblages de motifs chercher, et ainsi de suite. »

📊 **Exemple chiffré.** Notre modèle de départ, `yolov8n.pt`, contient **3 157 200 paramètres** et connaît 80 classes. Le modèle fine-tuné, `yolov8n_benin.pt`, en contient **3 011 628** et connaît 4 classes. [fait et vérifié] La différence de 145 572 paramètres vient de la dernière couche : sortir un score pour 4 classes demande moins de paramètres que pour 80. Vous pouvez le revérifier en une commande :

```
1 | .venv\Scripts\python.exe -c "from ultralytics import YOLO; m = YOLO('models/yolov8n.pt')
   | ; print(sum(x.numel() for x in m.model.parameters()), len(m.names))"
```

L'article annonce « 3,2 millions de paramètres » [article/sections/methode.tex:87-90], ce qui est cohérent avec la valeur mesurée de 3,157 million.

📁 **Chez nous.** Le fichier de poids est chargé en une ligne, à trois endroits : `scripts/train_yolo.py:52`, `scripts/evaluate.py:265` et `scripts/detect_image.py:56`. Un fichier `.pt` contient les paramètres appris ; c'est le seul livrable non régénérable sans relancer un entraînement complet, raison pour laquelle il est versionné par exception. [.gitignore:19-22]

⚠ **Piège et question de jury.** Croire que « plus de paramètres égale meilleur modèle ». Le modèle fine-tuné a *moins* de paramètres que le modèle de départ et le bat sur toutes les métriques. **Question de jury** : « pourquoi la version *nano* et pas une plus grande ? » Réponse : c'est un compromis assumé entre coût d'entraînement et représentativité, et c'est aussi la taille la plus plausible pour une inférence temps réel sur du matériel modeste. [article/sections/methode.tex:90-93]

3.2 YOLOv8 en trois blocs

🌱 **Analogie.** Une équipe de trois personnes examine une photo d'embouteillage. La première *résume* la photo à différentes échelles et jette les détails inutiles. La deuxième *fait circuler l'information* entre les échelles : ce que l'on voit de loin aide à interpréter ce que l'on voit de près, et réciproquement. La troisième *décide* : à cet endroit, il y a une moto, voici son rectangle, et j'en suis sûr à 0,87.

 **Définition, mot à mot.** Les trois blocs portent des noms consacrés.

Bloc	Nom courant	Rôle
1	Dorsale (<i>backbone</i>)	extrait des cartes d'activation de plus en plus abstraites, en réduisant progressivement la résolution
2	Cou (<i>neck</i>)	mélange les informations issues de plusieurs échelles, pour que les grands et les petits objets bénéficient chacun du contexte
3	Tête (<i>head</i>)	produit, pour chaque position candidate, une boîte, une classe et un score

Connaissance générale sur l'architecture YOLO ; le dépôt cite YOLOv8 et son implémentation Ultralytics [article/sections/related.tex:3-14] mais ne contient pas de schéma d'architecture. **absent du dépôt** pour tout schéma détaillé.

 **Exemple chiffré.** Le sous-échantillonnage successif explique mathématiquement notre problème central. Une image entre dans le réseau à 640 pixels de côté [configs/entrainement.yaml:11]. Après trois réductions par deux, la carte d'activation la plus fine fait 80 par 80 : une case y représente 8 pixels de l'image d'entrée. Un véhicule de 30 pixels sur 30 ne pèse donc plus que $30/8 \approx 4$ cases de côté environ, soit une quinzaine de cases au total. C'est peu d'information pour décider.

L'idée en une phrase

Voilà pourquoi les petits objets sont difficiles, et pourquoi ce n'est pas un défaut de programmation : après les réductions de résolution successives, un véhicule de moins de 32 sur 32 pixels ne pèse plus que quelques activations, souvent insuffisantes pour déclencher une détection. [article/sections/related.tex:16-26]

 **Piège et question de jury.** Dire « YOLO regarde l'image une seule fois donc il est moins précis ». La phrase exacte est : YOLO fait passer l'image *une seule fois* dans le réseau et prédit conjointement positions et classes, ce qui permet l'inférence temps réel indispensable à la vidéo-protection. [article/sections/related.tex:3-7] **Question de jury** : « pourquoi YOLOv8 et pas un détecteur à deux étages plus précis ? » Réponse : le cas d'usage est un flux vidéo de vidéoprotection, donc le débit est une contrainte de conception, pas un bonus ; et notre mesure montre 43,2 images par seconde pour le modèle fine-tuné. [results/comparaison.md:14]

3.3 Notion : le score de confiance

 **Analogie.** Un témoin qui dit « c'est une moto, j'en suis sûr à 90 pour cent ». Le nombre ne dit pas s'il a raison, il dit à quel point il s'engage.

 **Définition, mot à mot.** Le **score de confiance** est un nombre entre 0 et 1 attaché à chaque boîte prédite. On y applique un **seuil** : toute boîte dont le score est inférieur au seuil est jetée. Baisser le seuil produit plus de boîtes, donc plus de véhicules trouvés (meilleur rappel) mais aussi plus d'erreurs (moins bonne précision). Le monter fait l'inverse.

 **Exemple chiffré.** Le projet utilise deux seuils différents, et ce n'est pas une incohérence, c'est une nécessité :

Usage	Seuil	Pourquoi
Rappel par taille, visuels	0,25	seuil d'exploitation réaliste, celui qu'on utiliserait en production
mAP, précision, rappel globaux	0,001	il faut garder même les détections très peu sûres pour tracer la courbe précision-rappel en entier, dont la mAP est l'aire

[scripts/evaluate.py:43, 121 ; article/sections/methode.tex:181-184]

 **Chez nous.** SEUIL_CONFIAANCE = 0.25 est défini une seule fois, en `scripts/evaluate.py:43`, et réutilisé pour la mesure du rappel par taille et pour le débit. Le même 0,25 est la valeur par défaut de `detect_image.py:42`. Si vous le passiez à 0,10, le rappel des petits objets monterait mécaniquement et la comparaison resterait valide à condition de l'appliquer aux deux modèles, ce que le code garantit puisque les deux passent par la même fonction.

 **Piège et question de jury.** Comparer deux modèles à des seuils différents. C'est la faute la plus facile à commettre et la plus difficile à voir. **Question de jury :** « votre gain sur les petits objets ne vient-il pas simplement d'un seuil plus permissif ? » Réponse : non, le seuil 0,25 est codé une fois pour toutes et les deux modèles sont mesurés par la même fonction, appelée depuis le même script. [scripts/compare_models.py:19, 104-106]

3.4 Notion : l'IoU, le recouvrement

 **Analogie.** Deux personnes posent chacune un cadre photo sur la table pour encadrer le même objet. On mesure la surface commune aux deux cadres, puis la surface couverte par au moins un des deux. Le rapport des deux dit à quel point ils sont d'accord. Deux cadres parfaitement superposés donnent 1, deux cadres qui ne se touchent pas donnent 0.

 **Définition, mot à mot.** L'**IoU** (*Intersection over Union*, en français « intersection sur union ») compare deux boîtes A et B :

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

 **En français courant.** « L'aire de la partie commune aux deux rectangles, divisée par l'aire totale qu'ils recouvrent à eux deux. » [article/sections/methode.tex:198-201]

 **Exemple chiffré.** Prenons une boîte réelle de 100 par 100 pixels, coin haut gauche en (0,0), et une boîte prédite de 100 par 100 décalée de 20 pixels vers la droite et 20 vers le bas.

- Intersection : le rectangle commun va de (20,20) à (100,100), soit $80 \times 80 = 6\,400$ pixels carrés.
- Union : $10\,000 + 10\,000 - 6\,400 = 13\,600$ pixels carrés.
- IoU = $6\,400 / 13\,600 = 0,47$.

Avec le seuil du projet, fixé à 0,5, cette prédiction serait **rejetée** : décaler une boîte de 20 pour cent de sa taille suffit à la faire échouer. C'est sévère, et c'est voulu.

 **Chez nous.** Le calcul est fait à la main, sans bibliothèque, pour toutes les paires de boîtes d'une image à la fois :

```

149 x1 = np.maximum(boites_a[:, None, 0], boites_b[None, :, 0]) # bord gauche commun
150 y1 = np.maximum(boites_a[:, None, 1], boites_b[None, :, 1]) # bord haut commun
151 x2 = np.minimum(boites_a[:, None, 2], boites_b[None, :, 2]) # bord droit commun
152 y2 = np.minimum(boites_a[:, None, 3], boites_b[None, :, 3]) # bord bas commun
153
154 inter = np.clip(x2 - x1, 0, None) * np.clip(y2 - y1, 0, None) # 0 si pas de recouvrement
155 aire_a = (boites_a[:, 2] - boites_a[:, 0]) * (boites_a[:, 3] - boites_a[:, 1])
156 aire_b = (boites_b[:, 2] - boites_b[:, 0]) * (boites_b[:, 3] - boites_b[:, 1])
157 union = aire_a[:, None] + aire_b[None, :] - inter # union = somme moins commun
158 return np.where(union > 0, inter / union, 0.0) # jamais de division par zero

```

[scripts/evaluate.py:144-158]

Le `np.clip(..., 0, None)` est le détail qui compte : si les rectangles ne se croisent pas, $x_2 - x_1$ est négatif, et sans ce garde-fou on obtiendrait une intersection négative, donc un IoU négatif.

⚠ **Piège et question de jury.** Confondre les trois IoU du projet, qui n'ont rien à voir entre eux.

Où	Valeur	À quoi il sert
Rappel par taille	0,5	décider qu'une prédiction valide une boîte réelle [scripts/evaluate.py:44]
Valdateur Ultralytics	0,6	seuil de la suppression des non-maxima pendant la validation [scripts/evaluate.py:122]
Entraînement	0,7	même rôle, valeur par défaut pendant l'entraînement [results/entrainements/yolov8n_benin/args.yaml:43]
mAP@0,5 et mAP@0,5:0,95	0,5 puis 0,5 à 0,95	seuil d'appariement pour la métrique elle-même (section 5)

Question de jury : « pourquoi 0,5 et pas 0,75 pour votre rappel par taille ? » Réponse : 0,5 est la convention COCO pour une détection considérée comme correcte, et c'est le seuil le plus courant dans la littérature ; le choix est écrit une seule fois dans le code et s'applique aux deux modèles.

3.5 Notion : la suppression des non-maxima

🌱 **Analogie.** Dix personnes crient en même temps « la moto est là ! » en pointant presque le même endroit. On garde celle qui parle le plus fort et on fait taire toutes celles qui pointent au même endroit. Puis on recommence avec la plus forte des voix restantes.

🔧 **Définition, mot à mot.** La **suppression des non-maxima** (*Non-Maximum Suppression*, abrégé NMS) trie les boîtes prédites par score décroissant, garde la meilleure, supprime toutes celles qui la recouvrent trop (IoU au-dessus d'un seuil) et de même classe, puis recommence avec la suivante.

📊 **Exemple chiffré.** Supposons six boîtes prédites autour de la même moto, avec les scores 0,91 ; 0,88 ; 0,73 ; 0,64 ; 0,52 ; 0,31 et un IoU supérieur à 0,7 entre elles. Avec un seuil de NMS à 0,7, il reste une seule boîte, celle à 0,91. Sans NMS, le modèle annoncerait six motos là où il n'y en a qu'une, et la précision s'effondrerait.

📁 **Chez nous.** Le NMS est fait par Ultralytics à l'intérieur de `predict()` et de `val()` ; le projet ne le réimplémente pas. Son seuil vaut 0,7 à l'entraînement [results/entrainements/yolov8n_benin/args.yaml:43] et 0,6 lors de la validation appelée par notre code [scripts/evaluate.py:122]. Le nombre maximal de détections par image est de 300 [results/entrainements/yolov8n_benin/args.yaml:44], ce qui est important chez nous : nos images contiennent parfois plusieurs dizaines de véhicules.

⚠ **Piège et question de jury.** Le NMS est aussi une *cause* de faux négatifs en embouteillage : deux motos réellement collées l'une à l'autre peuvent avoir des boîtes qui se recouvrent à plus de 0,7, et la seconde est alors supprimée. **Question de jury :** « votre modèle rate encore 30 pour cent des petits objets, pourquoi ? » Réponse en trois volets : la limite informationnelle (sous un certain nombre de pixels, l'information n'existe plus dans l'image), le NMS qui fusionne des véhicules réellement superposés, et le seuil de confiance à 0,25 qui écarte les détections hésitantes.

[article/sections/discussion.tex:35-45]

3.6 Notion : la fonction de perte

 **Analogie.** C'est le barème de correction du devoir. Il transforme « ta réponse est fausse » en un nombre, et surtout il indique *dans quelle direction* corriger. Sans barème chiffré, impossible de progresser automatiquement.

 **Définition, mot à mot.** La **fonction de perte** (*loss*) mesure l'écart entre ce que le modèle prédit et ce qui est annoté. L'entraînement consiste à modifier les paramètres pour faire baisser ce nombre. YOLOv8 en utilise trois, additionnées avec des poids :

$$\mathcal{L} = 7,5 \times \mathcal{L}_{\text{boîte}} + 0,5 \times \mathcal{L}_{\text{classe}} + 1,5 \times \mathcal{L}_{\text{DFL}}$$

 **En français courant.** « L'erreur totale, c'est surtout l'erreur de placement du rectangle, un peu l'erreur d'étiquette, et un peu l'erreur sur la finesse des bords. »

Perte	Poids	Ce qu'elle pénalise
box_loss	7,5	un rectangle mal placé ou mal dimensionné
cls_loss	0,5	une moto annoncée comme une voiture
dfl_loss	1,5	une position de bord imprécise, la DFL (<i>Distribution Focal Loss</i>) traitant la coordonnée d'un bord comme une distribution de probabilité plutôt qu'un nombre unique

[results/entrainements/yolov8n_benin/args.yaml:84, 85, 87] **[fait et vérifié]**

 **Exemple chiffré.** Les trois pertes d'entraînement de notre run, de la première à la cinquantième epoch : la perte de boîte passe de 1,347 à 0,814, la perte de classification de 2,089 à 0,611, la perte DFL de 1,107 à 0,932. [results/entrainements/yolov8n_benin/results.csv:2, 51] **[fait et vérifié]** La classification est celle qui progresse le plus, ce qui se comprend : passer de 80 catégories à 4 catégories très différentes les unes des autres est une tâche plus simple que placer un rectangle au pixel près.

 **Chez nous.** Les poids 7,5 / 0,5 / 1,5 ne sont pas dans `configs/entrainement.yaml`, ce sont les valeurs par défaut d'Ultralytics ; elles sont *attestées* par le fichier `args.yaml` que le run a écrit, ce qui est précisément la raison pour laquelle ce fichier est versionné par exception. [.gitignore:37-42]

 **Piège et question de jury.** Croire qu'une perte faible signifie un bon modèle. La perte est une grandeur interne, sans unité interprétable : 0,81 ne veut rien dire dans l'absolu. Seule sa *tendance* compte, et seules les métriques de la section 5 sont interprétables. **Question de jury** : « votre perte de boîte vaut 0,81, est-ce bon ? » Réponse : la question ne se pose pas ainsi ; ce qui compte est qu'elle décroisse de façon monotone et que la perte de validation ne remonte pas, ce qui est notre cas jusqu'à la dernière epoch.

À retenir

Le modèle en une phrase : un empilement de filtres appris (3,16 million de paramètres pour `yolov8n.pt`), organisé en dorsale, cou et tête, qui produit des boîtes avec un score, nettoyées par la suppression des non-maxima, et entraîné en faisant baisser une somme pondérée de trois pertes.

4 L'entraînement

4.1 Notion : le transfert d'apprentissage

 **Analogie.** Un mécanicien qui a réparé des voitures européennes pendant dix ans arrive à Cotonou. Il ne réapprend pas ce qu'est un moteur, une roue ou un frein : il ajuste ses réflexes aux modèles locaux, aux routes, aux pannes fréquentes. Trois semaines suffisent là où une formation complète prendrait trois ans.

 **Définition, mot à mot.** Le **transfert d'apprentissage** (*transfer learning*) consiste à partir d'un modèle déjà entraîné sur un grand jeu de données généraliste, puis à poursuivre l'entraînement sur un petit jeu spécialisé. Le **fine-tuning** (« affinage ») est la variante où l'on continue d'ajuster *tous* les paramètres, et non seulement les dernières couches.

 **En français courant.** « On ne repart pas de zéro : on reprend un modèle qui sait déjà voir des formes, et on lui apprend nos routes. »

 **Exemple chiffré.** Notre point de départ est `yolov8n.pt`, entraîné sur COCO, qui contient 80 catégories du quotidien. Nous poursuivons son entraînement 50 epochs sur 2400 images de trafic dense. Le résultat, mesuré sur le même jeu de test : la mAP@0,5 passe de 0,438 à 0,825. [results/comparaison.md:9]

 **Chez nous.** Le transfert tient en une ligne de code, la ligne 52 de `train_yolo.py` :

```
52     modele = YOLO(str(modele_base)) # on CHARGE les poids COCO, on ne cree pas un reseau vide
53     resultats = modele.train(
54         data=str(donnees),
55         epochs=args.epochs or config["epochs"],
56         ...
57     )
```

[scripts/train_yolo.py:52-66] Le fichier `args.yaml` écrit par le run confirme `pretrained: true` et `model: .../yolov8n.pt`. [results/entraînements/yolov8n_benin/args.yaml:3, 18]

 **Piège et question de jury.** Croire qu'un fine-tuning « ajoute » des connaissances sans en perdre. Le modèle fine-tuné ne connaît plus que 4 classes : il ne sait plus détecter un chien ou une chaise, et c'est parfaitement voulu ici. **Question de jury** : « pourquoi ne pas avoir entraîné depuis zéro ? » Réponse : avec 2400 images, un entraînement depuis zéro donnerait un modèle très inférieur ; l'article observe d'ailleurs que l'essentiel de l'adaptation se joue dans le premier tiers de l'entraînement, ce qui est cohérent avec l'idée que le fine-tuning réutilise des représentations déjà formées. [article/sections/discussion.tex:104-115 (voir résultats)]

4.2 Chaque hyperparamètre, expliqué

4.2.1 Notion : hyperparamètre

 **Analogie.** Les paramètres, ce sont les réglages que la machine trouve toute seule en s'entraînant. Les hyperparamètres, ce sont les boutons que *vous* tournez avant d'appuyer sur démarrer : durée, taille des lots, intensité des transformations.

 **Définition, mot à mot.** Un **hyperparamètre** est un réglage fixé avant l'entraînement et qui n'est pas appris. Le principe de conception du projet est qu'aucun hyperparamètre n'est écrit en dur dans le code : ils vivent tous dans `configs/entraînement.yaml`, de sorte qu'un run soit entièrement décrit par son fichier de configuration, donc reproductible. [scripts/train_yolo.py:3-6 ; docs/STRUCTURE.md:51-53]

4.2.2 Le fichier, ligne par ligne

Ligne	Réglage	Ce qu'il fait, et l'effet si on le change
5	<code>modele_base: models/yolov8n.pt</code>	Le modèle de départ. Le commentaire suggère <code>yolov8s</code> sur Colab ; le run réel a bien utilisé <code>yolov8n</code> . Passer à <code>yolov8s</code> augmenterait la précision et diviserait environ le débit par deux.
8	<code>donnees: configs/dataset.yaml</code>	Le fichier qui dit à Ultralytics où sont les images et quelles sont les classes. Il est régénéré à chaque import, ne pas l'éditer à la main.
11	<code>epochs: 50</code>	Nombre de passages complets sur les 2 400 images. Doubler à 100 augmenterait probablement encore les métriques, puisque nous n'avions pas convergé (voir plus bas), au prix d'une heure de calcul supplémentaire.
12	<code>taille_image: 640</code>	Côté de l'image redimensionnée en entrée du réseau. Le réglage le plus sensible pour notre sujet : baisser à 416 accélérerait mais détruirait les petits objets ; monter à 960 les aiderait mais multiplierait le temps de calcul par environ 2,25.
13	<code>batch: 16</code>	Nombre d'images traitées ensemble avant chaque mise à jour des paramètres. C'est d'abord une contrainte de mémoire GPU. Un batch plus grand donne des mises à jour plus stables mais moins nombreuses.
14	<code>patience: 15</code>	Arrêt anticipé : si aucune amélioration n'est constatée pendant 15 epochs consécutives, l'entraînement s'arrête. Chez nous il ne s'est jamais déclenché.
15	<code>device: ""</code>	Vide laisse Ultralytics choisir, <code>"cpu"</code> force le processeur, <code>"0"</code> force le premier GPU. Le run réel s'est fait avec <code>device: '0'</code> .
16	<code>workers: 4</code>	Nombre de processus qui préparent les images en parallèle. N'influence que la vitesse, jamais le résultat.
17	<code>seed: 42</code>	Graine aléatoire. Indispensable pour comparer deux runs : sans elle, deux entraînements identiques donneraient des chiffres légèrement différents.
20	<code>projet: results/entraitements</code>	Dossier où Ultralytics écrit le run.
21	<code>nom_run: yolov8n_benin</code>	Nom du sous-dossier, et nom du fichier de poids copié dans <code>models/finetuned/</code> .

[configs/entrainement.yaml:1-21 ; results/entraitements/yolov8n_benin/args.yaml:3, 5, 8-9, 13-14, 22] **[fait et vérifié]**

4.2.3 Les réglages que nous n'avons pas choisis, mais qui ont agi

Ultralytics applique ses valeurs par défaut à tout ce que le fichier ne précise pas. Le fichier `args.yaml` du run les atteste, et un jury technique peut les demander.

Réglage	Valeur	Sens
optimizer	auto	l'algorithme de mise à jour est choisi automatiquement
lr0	0,01	taux d'apprentissage initial, c'est-à-dire la taille du pas de correction
lrf	0,01	le taux final vaut $0,01 \times$ le taux initial : le pas rétrécit au fil du temps
momentum	0,937	inertie : la correction garde une partie de la direction précédente
weight_decay	0,0005	pénalise les paramètres trop grands, contre le surapprentissage
warmup_epochs	3,0	les trois premières epochs montent progressivement en régime
close_mosaic	10	la mosaïque est désactivée pendant les 10 dernières epochs
amp	true	calcul en précision mixte, plus rapide sur GPU
nbs	64	taille de lot nominale servant à normaliser le taux d'apprentissage
max_det	300	au plus 300 détections par image

[results/entrainements/yolov8n_benin/args.yaml:20, 27, 44, 75-81, 95] **[fait et vérifié]**

L'idée en une phrase

close_mosaic: 10 explique une bizarrerie visible sur nos courbes : entre l'epoch 40 et l'epoch 41, la perte de boîte d'entraînement chute brutalement de 0,937 à 0,880, et la perte de classification de 0,738 à 0,694. [results/entrainements/yolov8n_benin/results.csv:41-42] **[fait et vérifié]** Ce n'est pas un progrès du modèle : c'est que les dix dernières epochs se font sur des images normales, plus faciles que les mosaïques. Savoir expliquer ce décrochage est un excellent point devant un jury, et il n'est pas commenté dans l'article.

4.3 Notion : l'augmentation de données

 **Analogie.** Pour préparer un examen avec seulement dix exercices, vous les recopiez en changeant les nombres, en tournant la feuille, en photocopiant en plus sombre. Vous n'avez pas plus de connaissances, mais vous devenez robuste aux présentations différentes.

 **Définition, mot à mot.** L'**augmentation de données** transforme aléatoirement chaque image d'entraînement à chaque passage : découpage, zoom, symétrie, changement de couleur. Le modèle ne voit donc jamais exactement deux fois la même image, ce qui réduit le surapprentissage et le rend robuste aux variations réelles. **Elle ne s'applique qu'à l'entraînement**, jamais à la validation ni au test.

 **Exemple chiffré.** Nos neuf réglages, et surtout leur justification :

Réglage	Valeur	Effet et justification
mosaic	1,0	Assemble quatre images en une , systématiquement. Chaque véhicule occupe donc environ quatre fois moins de surface : c'est une fabrique à petits objets, et c'est l'augmentation la plus directement liée à notre problématique.
scale	0,7	Zoom aléatoire de plus ou moins 70 pour cent. Un zoom arrière produit lui aussi des objets plus petits que ceux réellement photographiés.
flip_lr	0,5	Symétrie gauche-droite une fois sur deux. Plausible : une voiture vue de l'autre côté de la rue existe.
flipud	0,0	Retournement vertical désactivé : une scène routière a un haut et un bas, une voiture au plafond n'existe pas.
degrees	0,0	Rotation désactivée : les caméras de vidéoprotection sont fixes.
translate	0,1	Déplacement de l'image jusqu'à 10 pour cent, simule un cadrage légèrement différent.
hsv_h	0,015	Variation de teinte, très faible : une voiture rouge doit rester rouge.
hsv_s	0,7	Variation de saturation, forte : couleurs délavées par le soleil.
hsv_v	0,4	Variation de luminosité : plein soleil contre zone d'ombre.

[configs/entrainement.yaml:27-37 ; article/sections/methode.tex:118-133 ; valeurs confirmées par results/entrainements/yolov8n_benin/args.yaml:96-107] **[fait et vérifié]**

📁 **Chez nous.** Les réglages sont transmis au moteur d'entraînement par un simple dépliage de dictionnaire, ce qui garantit qu'aucune valeur n'est retapée ailleurs :

```
39 augmentation = config.get("augmentation", {})
40 ...
41 **augmentation, # les 9 réglages du YAML sont passés tels quels
```

[scripts/train_yolo.py:39, 65]

⚠ Vérification dans le dépôt

Le fichier `args.yaml` enregistre aussi `erasing: 0.4` et `auto_augment: randaugment` [results/entrainements/yolov8n_benin/args.yaml:111-113]. Ce sont des valeurs par défaut d'Ultralytics. Le dépôt ne documente ni ne discute leur effet sur une tâche de détection : si un membre du jury les mentionne, la réponse honnête est « ce sont des valeurs par défaut que nous n'avons pas modifiées, et que nous n'avons pas étudiées ».

⚠ **Piège et question de jury.** Croire que l'augmentation « crée des données ». Elle ne crée aucune information nouvelle : elle recycle la même information sous des présentations différentes. Si vos 2 400 images ne contiennent aucun bus de nuit, aucune augmentation n'en fabriquera. **Question de jury :** « comment savez-vous que mosaic et scale sont responsables du gain sur les petits objets ? » Réponse honnête, et c'est celle que l'article donne : ce sont les explications les plus plausibles, mais notre protocole ne permet pas d'isoler la contribution de chacune ; il faudrait une étude d'ablation, c'est-à-dire réentraîner en désactivant un réglage à la fois. [article/sections/discussion.tex:16-20]

4.4 Lire les courbes d'entraînement

4.4.1 Ce qu'est une epoch

🌱 **Analogie.** Une epoch, c'est une révision complète du cahier d'exercices, du premier au dernier. Cinquante epochs, c'est cinquante relectures, à chaque fois dans un ordre différent et avec des variantes.

🔧 **Définition, mot à mot.** Une **epoch** est un passage complet sur l'ensemble du jeu d'entraînement. Avec 2 400 images et un batch de 16, une epoch compte $2\,400/16 = 150$ mises à jour des paramètres. Cinquante epochs font donc 7 500 mises à jour.

📊 **Exemple chiffré.** Notre run a duré 3 293,76 secondes au total, soit 54,9 minutes, pour 50 epochs sur un GPU T4, c'est-à-dire environ 66 secondes par epoch. [results/entrainements/yolov8n_benin/results.csv:51 ; article/sections/discussion.tex:135] **[fait et vérifié]**

4.4.2 Les quatre courbes qui comptent

Le fichier `results.csv` contient 15 colonnes et 50 lignes, une par epoch. [results/entrainements/yolov8n_benin/results.csv:1]

Colonne	Ce qu'on y cherche
train/box_loss et compagnie	doivent descendre ; une remontée signale un problème de réglage
val/box_loss et compagnie	doivent descendre <i>aussi</i> ; si elles remontent alors que celles d'entraînement descendent, c'est du surapprentissage
metrics/mAP50(B)	la qualité de détection sur la validation, doit monter puis se stabiliser
metrics/mAP50-95(B)	la même chose, en plus exigeant ; c'est la courbe la plus informative

4.4.3 Ce que nos courbes racontent, chiffres à l'appui

1. **Une convergence rapide, puis un plateau.** La mAP@0,5:0,95 de validation vaut 0,406 après la première epoch et 0,669 à la cinquantième, avec un maximum de 0,670 à l'epoch 47. Elle franchit dès l'epoch 29 le seuil de 95 pour cent de sa valeur finale : les vingt et une dernières epochs n'apportent que les 5 pour cent restants. [results/entrainements/yolov8n_benin/results.csv:2, 30, 48, 51 ; article/sections/resultats.tex:104-115] [\[fait et vérifié\]](#)
2. **Un décrochage à l'epoch 2**, où la mAP@0,5 recule de 0,591 à 0,534. Ce n'est pas une anomalie : c'est la montée en puissance du taux d'apprentissage pendant les trois epochs d'échauffement (warmup_epochs: 3). Le décrochage se résorbe dès l'epoch 4. [\[fait et vérifié\]](#)
3. **Aucun surapprentissage à 50 epochs.** Les pertes de validation atteignent leur minimum en toute fin d'entraînement : epoch 50 pour la boîte, 49 pour la classification, 46 pour la DFL. [article/sections/resultats.tex:117-127] [\[fait et vérifié\]](#)
4. **L'arrêt vient du budget, pas de la convergence.** La patience de 15 epochs n'a jamais été déclenchée. Les 50 epochs sont une borne que nous avons imposée, pas un point d'arrêt que les données auraient dicté. Nos chiffres sont donc un **plancher**, et non un plafond. [article/sections/resultats.tex:129-137]

Devant le jury

La question qui fait mouche, et vous avez la réponse : « votre entraînement avait-il convergé ? »

Réponse en trois phrases : non, et nous le disons. Les trois pertes de validation sont encore à leur minimum à la dernière epoch et l'arrêt anticipé ne s'est jamais déclenché, donc la borne de 50 epochs a été atteinte avant la convergence. Nos chiffres absolus sont un plancher, et prolonger l'entraînement est la première expérience à mener.

4.5 Notion : le surapprentissage

 **Analogie.** Un élève qui apprend par cœur les corrigés au lieu de comprendre la méthode. Il a 20 sur 20 au cahier d'exercices et 6 sur 20 à l'examen.

 **Définition, mot à mot.** Le **surapprentissage** (*overfitting*) survient quand le modèle mémorise les particularités du jeu d'entraînement au lieu d'apprendre des régularités transposables. Le signe visible : la perte d'entraînement continue de baisser pendant que la perte de *validation* remonte, et l'écart entre les deux se creuse.

 **Exemple chiffré.** Voici les valeurs de notre run, aux deux bouts :

Perte	Epoch 1	Epoch 50	Minimum atteint à
train/box	1,347	0,814	epoch 49
val/box	1,083	0,834	epoch 50
train/cls	2,088	0,611	epoch 50
val/cls	1,325	0,659	epoch 49
train/dfll	1,107	0,932	epoch 49
val/dfll	1,027	0,955	epoch 46

[results/entrainements/yolov8n_benin/results.csv] [\[fait et vérifié\]](#)

Les pertes de validation ne remontent jamais durablement : il n'y a pas de surapprentissage.

 **Chez nous.** Trois garde-fous étaient en place et n'ont pas eu à servir : l'arrêt anticipé (patience: 15, configs/entrainement.yaml:14), la pénalisation des poids (weight_decay: 0.0005, args.yaml:78) et l'augmentation de données, qui est en pratique la protection la plus efficace des trois sur un petit jeu.

⚠ **Piège et question de jury.** Confondre « pertes de validation basses » et « modèle bon ». Un modèle peut parfaitement ne pas surapprendre *et* être médiocre, s'il sous-apprend. **Question de jury :** « comment auriez-vous vu le surapprentissage arriver ? » Réponse : par le croisement des courbes de perte, visible sur la figure des courbes d'entraînement, et par le déclenchement de l'arrêt anticipé, qui aurait coupé l'entraînement 15 epochs après le dernier progrès.

✦ À retenir

L'entraînement en une phrase : on part des poids COCO, on repasse 50 fois sur 2 400 images fortement augmentées vers les petits objets, en 55 minutes sur GPU T4, et on s'arrête avant la convergence faute de budget, ce qui fait de tous nos chiffres un plancher.

5 L'évaluation

5.1 Notion : vrai positif, faux positif, faux négatif

 **Analogie.** Un vigile à l'entrée d'un stade doit repérer les personnes sans billet. Il en arrête une sans billet : c'est un **vrai positif**. Il en arrête une qui avait son billet : **faux positif**. Il en laisse passer une sans billet : **faux négatif**. Un vigile qui n'arrête personne ne fait aucun faux positif, et rate tout le monde.

 **Définition, mot à mot.** En détection, la décision se prend par appariement entre une boîte prédite et une boîte annotée, à l'aide de l'IoU.

Sigle	Définition
VP (vrai positif)	une boîte prédite qui recouvre suffisamment une boîte annotée, avec la bonne classe
FP (faux positif)	une boîte prédite qui ne correspond à aucune boîte annotée, ou une seconde boîte sur un objet déjà trouvé
FN (faux négatif)	une boîte annotée qu'aucune prédiction n'a couverte, autrement dit un véhicule manqué

Il n'y a pas de « vrai négatif » utile en détection : le nombre de rectangles où il n'y a rien est pratiquement infini.

 **Exemple chiffré.** Sur les 804 petits objets de notre jeu de test, le modèle pré-entraîné en apparie 98. Donc VP = 98 et FN = 804 – 98 = 706. Le modèle fine-tuné en apparie 558, donc VP = 558 et FN = 246. [results/comparaison.json:15-18, 45-48] **[fait et vérifié]**

 **Chez nous.** L'appariement du rappel par taille est fait à la main, avec une règle explicite : une prédiction ne peut valider qu'une seule boîte réelle.

```

216     deja_prises: set[int] = set() # predictions deja utilisees
217     for i in range(len(reelles)):
218         ...
219         ordre = np.argsort(-ious[i]) # predictions trieées par recouvrement décroissant
220         for j in ordre:
221             if ious[i, j] < SEUIL_IOU: # plus rien d'assez recouvrant : on abandonne
222                 break
223             if j not in deja_prises: # une prediction ne sert qu'une fois
224                 deja_prises.add(int(j))
225                 compte[bucket]["detecte"] += 1
226                 break

```

[scripts/evaluate.py:215-236]

⚠ Vérification dans le dépôt

Deux précisions d'honnêteté sur ce calcul, que le dépôt assume et qu'il vaut mieux annoncer soi-même. **Un** : cet appariement est *glouton* et parcourt les boîtes réelles dans l'ordre du fichier, pas dans l'ordre du meilleur recouvrement global ; dans de rares cas, un appariement globalement optimal donnerait un ou deux objets de plus. [article/sections/methode.tex:162-168] **Deux** : la mesure est **agnostique à la classe**, c'est-à-dire qu'une voiture détectée comme un camion compte quand même comme détectée. C'est écrit dans l'article [article/sections/methode.tex:168] et c'est cohérent avec la question posée : « le véhicule a-t-il été vu ? », et non « a-t-il été correctement nommé ? ». Le même choix s'applique aux deux modèles.

 **Piège et question de jury.** Croire qu'un faux positif est moins grave qu'un faux négatif. Cela dépend entièrement de l'usage : pour un comptage de trafic, un faux positif gonfle le chiffre ; pour

une verbalisation automatique, un faux positif verbalise un innocent. **Question de jury** : « quel type d'erreur vous inquiète le plus dans un système de vidéo-verbalisation ? » Réponse : le faux positif, car il produit une sanction injustifiée ; c'est d'ailleurs une raison de plus pour que la lecture de plaques et les règles d'infraction soient hors de notre périmètre.

5.2 Notion : précision, rappel, F1

 **Analogie.** La précision répond à « quand j'accuse quelqu'un, ai-je raison ? ». Le rappel répond à « parmi tous les coupables, combien en ai-je attrapé ? ». On peut avoir une précision parfaite en n'accusant qu'une seule personne dont on est certain, et un rappel parfait en accusant tout le monde.

 **Définition, mot à mot.** Deux formules, et leur traduction.

$$\text{Précision} = \frac{VP}{VP + FP} \qquad \text{Rappel} = \frac{VP}{VP + FN}$$

 **En français courant.** Précision : « sur 100 véhicules que j'annonce, combien existent vraiment ? ». Rappel : « sur 100 véhicules réellement présents, combien ai-je trouvés ? ».
 [article/sections/methode.tex:186-192]

Le **F1-score** est la moyenne harmonique des deux :

$$F_1 = 2 \cdot \frac{\text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}}$$

 **En français courant.** « Une note unique qui punit sévèrement le déséquilibre : si l'une des deux est nulle, le F1 est nul, quelle que soit l'autre. »

 **Exemple chiffré.** Refaisons le calcul du F1 de notre modèle fine-tuné, à la main, avec les vrais nombres :

$$F_1 = 2 \times \frac{0,8327 \times 0,7138}{0,8327 + 0,7138} = 2 \times \frac{0,5944}{1,5465} = \frac{1,1888}{1,5465} = 0,7687$$

C'est exactement la valeur du tableau. [results/comparaison.md:13] **[fait et vérifié]** Même calcul pour le modèle pré-entraîné : $2 \times (0,4959 \times 0,4172) / (0,4959 + 0,4172) = 0,4532$.

Interprétation en français : le modèle pré-entraîné se trompe une fois sur deux quand il annonce un véhicule, et il en rate près de six sur dix. Le modèle fine-tuné a raison plus de huit fois sur dix et en trouve plus de sept sur dix.

 **Chez nous.** Le F1 n'est pas fourni par la bibliothèque, il est recalculé par notre code, ce qui explique qu'il soit exactement cohérent avec les deux autres colonnes :

```
127 | precision = float(boite.mp) # moyenne des precisions par classe
128 | rappel = float(boite.mr) # moyenne des rappels par classe
129 | f1 = 2 * precision * rappel / (precision + rappel) if (precision + rappel) else 0.0
```

[scripts/evaluate.py:127-130]

Vérification dans le dépôt

Nuance importante, et un jury pointilleux peut la soulever. La précision et le rappel du tableau ne sont **pas** mesurés au seuil de confiance 0,25 : ce sont les valeurs **mp** et **mr** du validateur d'Ultralytics, calculées sur la courbe précision-rappel construite avec un seuil de confiance de 0,001. [scripts/evaluate.py:121, 127-128] Elles ne sont donc pas directement comparables au rappel par taille, qui est mesuré à 0,25. Les deux mesures restent valides ; elles répondent simplement à deux questions différentes, et ce qui compte est que le *même* protocole soit appliqué aux deux modèles.

 **Piège et question de jury.** Annoncer une précision de 0,83 comme « le modèle est juste à 83 pour cent ». Une précision élevée obtenue avec un rappel faible décrit un modèle timide, pas un bon

modèle. **Question de jury** : « votre rappel de 0,714 est plus faible que votre précision de 0,833, pourquoi ? » Réponse : parce que notre jeu contient énormément de très petits objets, et qu'il reste plus facile de ne pas se tromper sur ce que l'on voit que de tout voir.

5.3 Notion : la courbe précision-rappel

 **Analogie.** Vous tournez lentement le bouton de sensibilité du vigile, du plus méfiant au plus laxiste. À chaque cran, vous notez le couple (précision, rappel). En reliant les points, vous obtenez une courbe qui décrit le *comportement complet* du modèle, et non une seule de ses humeurs.

 **Définition, mot à mot.** La **courbe précision-rappel** porte le rappel en abscisse et la précision en ordonnée. Chaque point correspond à un seuil de confiance. Un bon modèle garde une précision élevée même quand le rappel augmente : sa courbe reste haute, puis tombe tard et brutalement vers la droite. Un mauvais modèle voit sa précision chuter dès les premiers pourcents de rappel.

 **Exemple chiffré.** Sur la courbe produite par notre run, l'aire sous la courbe vaut 0,910 pour la voiture, 0,868 pour la moto, 0,833 pour le bus, 0,760 pour le camion, et 0,843 toutes classes confondues. [results/entrainements/yolov8n_benin/BoxPR_curve.png] [\[fait et vérifié\]](#)

 **Chez nous.** C'est pour tracer cette courbe en entier que le seuil de confiance est descendu à 0,001 lors du calcul des métriques globales : sans les détections peu sûres, la partie droite de la courbe manquerait et l'aire serait sous-estimée. Le commentaire du code le dit en une ligne : « convention de détection : seuil bas pour la courbe P/R ». [scripts/evaluate.py:121]

 **Piège et question de jury.** Lire la courbe comme une courbe de progression dans le temps. Elle ne décrit pas l'entraînement, elle décrit un modèle figé à un instant donné. **Question de jury** : « que représente un point de cette courbe ? » Réponse : un choix de seuil de confiance, et donc un compromis d'exploitation.

5.4 Notion : la mAP

 **Analogie.** C'est la moyenne des notes obtenues par chaque classe, chaque note étant l'aire sous sa courbe précision-rappel. Une seule note pour tout le modèle, mais construite honnêtement, classe par classe.

 **Définition, mot à mot.** **AP** signifie *Average Precision*, précision moyenne : c'est l'aire sous la courbe précision-rappel d'une classe. **mAP** signifie *mean Average Precision* : la moyenne des AP sur toutes les classes.

$$\text{mAP} = \frac{1}{N} \sum_{i=1}^N \text{AP}_i$$

 **En français courant.** « On calcule une note par classe, puis on fait la moyenne des notes. » [article/sections/methode.tex:203-208]

Deux variantes existent, et la différence est uniquement le seuil d'IoU utilisé pour dire qu'une détection est correcte :

Métrique	Sens
mAP@0,5	une seule exigence de recouvrement, permissive : IoU au moins 0,5
mAP@0,5:0,95	la moyenne des mAP obtenues à dix seuils, de 0,5 à 0,95 par pas de 0,05 ; il faut donc être bon <i>et</i> précis dans le placement du rectangle

[README.md:196-197 ; article/sections/methode.tex:177-179]

 **Exemple chiffré.** Nos deux modèles, sur le jeu de test :

Métrique	Pré-entraîné	Fine-tuné	Rapport
mAP@0,5	0,4380	0,8252	×1,88
mAP@0,5:0,95	0,2959	0,6443	×2,18

[results/comparaison.md:9-10] **[fait et vérifié]**

Le fait que la mAP@0,5:0,95 progresse *plus* que la mAP@0,5 est instructif : le fine-tuning n'a pas seulement appris à voir davantage de véhicules, il a aussi appris à placer les rectangles plus finement. C'est l'effet direct de la perte de boîte et de la perte DFL, qui ont diminué tout au long de l'entraînement.

📁 Chez nous. Les deux valeurs sortent des attributs `map50` et `map` de l'objet renvoyé par le validateur. [scripts/evaluate.py:132-133]

⚠ Piège et question de jury. Comparer une mAP à une autre publiée ailleurs. Une mAP n'a de sens que sur un jeu de test donné, avec un protocole donné. Les 97,9 % cités pour le travail ougandais concernent la détection de *plaques* sur *leur* jeu, et ne sont pas comparables aux nôtres. [article/sections/related.tex:41-48] **Question de jury** : « votre mAP de 0,825 est-elle bonne ? » Réponse : elle n'a de sens que relativement au 0,438 du même modèle avant fine-tuning, sur exactement le même jeu de test ; c'est l'écart qui est notre résultat, pas la valeur absolue.

5.5 Notion : la matrice de confusion

🌱 Analogie. Un tableau à double entrée qui dit, pour chaque type de véhicule réellement présent, ce que le modèle a répondu. Les bonnes réponses sont sur la diagonale, les confusions ailleurs.

🔧 Définition, mot à mot. La **matrice de confusion** croise la vérité en colonnes et la prédiction en lignes. Une ligne et une colonne supplémentaires, appelées *background* (« arrière-plan »), servent aux détections qui ne correspondent à rien et aux objets que rien n'a couverts. Dans sa version normalisée, chaque colonne est ramenée à un total de 1, donc les valeurs se lisent comme des proportions.

📊 Exemple chiffré. La matrice de notre run montre, en diagonale : 0,72 pour la voiture, 0,89 pour la moto, 0,49 pour le bus, 0,54 pour le camion. Hors diagonale, les deux confusions notables sont bus prédit comme voiture (0,21) et camion prédit comme bus (0,23), ainsi que bus prédit comme camion (0,24). [results/entrainements/yolov8n_benin/confusion_matrix_normalized.png] **[fait et vérifié]**

💡 L'idée en une phrase

Un bus et un camion, vus par une caméra de vidéoprotection, sont deux grands volumes rectangulaires. La confusion entre les deux est la plus fréquente de notre matrice, et elle est *sémantique*, pas *géométrique* : le véhicule est bien détecté, il est mal nommé. C'est exactement pour cela que notre mesure principale, le rappel par taille, est agnostique à la classe.

⚠ Vérification dans le dépôt

Attention à la lecture de la colonne *background* : elle vaut 0,53 pour la moto, ce qui signifie qu'une large part des détections sans correspondance sont des motos annoncées là où rien n'est annoté. Cette matrice est produite par Ultralytics pendant la validation, avec un seuil de confiance très bas : elle compte donc beaucoup de détections hésitantes qui n'apparaîtraient pas au seuil d'exploitation de 0,25. **Elle porte de plus sur le jeu de validation, pas sur le jeu de test** : les 300 images de validation, et non les 300 de test. Ne la présentez pas comme illustrant le tableau comparatif.

⚠ Piège et question de jury. Lire la matrice dans le mauvais sens. Ici les colonnes sont la vérité et les lignes la prédiction ; l'inverse existe dans d'autres outils. **Question de jury** : « votre

matrice montre 0,49 pour le bus, votre modèle est donc mauvais sur les bus ? » Réponse : le bus est notre classe la moins représentée, avec 1 298 instances d'entraînement contre 11 641 pour la moto, et sa principale confusion est le camion, qui lui ressemble ; nous ne publions volontairement pas de métrique par classe pour cette raison.

5.6 Notion : le débit, ou FPS

 **Analogie.** Combien de photos par seconde le système peut-il traiter. Une caméra qui produit 25 images par seconde exige un système capable de suivre, sinon il faut sauter des images.

 **Définition, mot à mot.** Le **FPS** (*frames per second*, images par seconde) est le nombre d'images traitées par seconde. Il se calcule comme le nombre d'images divisé par le temps total.

 **Exemple chiffré.** Nos mesures : 37,2 FPS pour le modèle pré-entraîné, 43,2 FPS pour le modèle fine-tuné, soit +16,2 %. [results/comparaison.md:14]

 **Chez nous.** La mesure est faite par notre propre code, sur les mêmes images que le reste, avec une précaution explicite : une inférence d'échauffement est lancée et *exclue* du chronomètre, car le premier appel paie l'initialisation du modèle et de la carte graphique.

```

190     # Echauffement : le premier appel paie l'initialisation du modele et du
191     # device ; il ne doit pas entrer dans la mesure du debit.
192     modele.predict(str(images[0]), conf=SEUIL_CONFIANCE, classes=classes,
193                   device=device, verbose=False)
194
195     for chemin in images:
196         debut = time.perf_counter()
197         prediction = modele.predict(...)[0]
198         duree_totale += time.perf_counter() - debut

```

[scripts/evaluate.py:190-204, 246]

⚠ Vérification dans le dépôt

Trois limites de cette mesure, à connaître avant qu'on vous les oppose. **Un** : le chronomètre englobe la lecture du fichier image, le redimensionnement, l'inférence et la suppression des non-maxima. Ce n'est donc pas un débit de réseau pur, c'est un débit de bout en bout image par image, ce qui est plus honnête pour un cas d'usage mais plus lent qu'un traitement par lots. **Deux** : le matériel utilisé n'est enregistré nulle part dans les fichiers de résultats. L'article annonce un GPU T4 [article/sections/resultats.tex:11], ce qui est cohérent avec le notebook Colab, mais aucun fichier du dépôt ne l'atteste pour la comparaison. **[fait mais non vérifiable dans le dépôt]** **Trois** : l'écart de +16,2 % ne peut pas venir de l'architecture, qui est identique. L'article avance l'hypothèse d'une suppression des non-maxima allégée par le passage de 80 à 4 classes, plus la variabilité d'un environnement partagé comme Colab, et reconnaît n'avoir pas conçu d'expérience pour départager les deux. [article/sections/discussion.tex:47-53]

 **Piège et question de jury.** Annoncer un FPS sans dire sur quel matériel. Un même modèle peut faire 40 images par seconde sur GPU et 3 sur un processeur d'ordinateur portable. **Question de jury** : « votre modèle tient-il le temps réel ? » Réponse : 43 images par seconde sur GPU T4 pour une caméra à 25 images par seconde, donc oui sur ce matériel, avec la réserve que nous n'avons pas mesuré sur une carte embarquée, qui serait le matériel réel d'un déploiement.

5.7 Notion : le rappel par taille d'objet

💡 L'idée en une phrase

C'est la mesure propre au sujet, celle qui rattache les chiffres au titre du projet. Une moyenne globale masque précisément l'effet que nous cherchons : un gain de mAP pourrait très bien provenir des seuls gros véhicules au premier plan, auquel cas le projet n'aurait rien résolu.

[README.md:203-206]

🌱 **Analogie.** Un professeur qui annonce « la classe a progressé de 3 points de moyenne » sans dire si ce sont les bons élèves ou les élèves en difficulté qui ont progressé. Le rappel par taille, c'est la moyenne ventilée par niveau.

🔧 **Définition, mot à mot.** On range chaque véhicule annoté dans l'une de trois catégories selon l'**aire** de sa boîte, en suivant la convention du benchmark COCO :

Catégorie	Condition sur l'aire	En pixels carrés
petit	$\text{aire} < 32 \times 32$	moins de 1 024
moyen	$32 \times 32 \leq \text{aire} < 96 \times 96$	de 1 024 à 9 216
grand	$\text{aire} \geq 96 \times 96$	9 216 et plus

[scripts/evaluate.py:46-51]

Puis on calcule, dans chaque catégorie, le rappel : combien de ces objets ont été retrouvés.

5.7.1 Le détail qui sauve la mesure : la normalisation des aires

📊 **Exemple chiffré.** Voici pourquoi ce détail est décisif. Prenons une image de 1 920 pixels de large et une petite voiture au fond, de 60 pixels sur 45. Son aire vaut $60 \times 45 = 2\,700$ pixels carrés : elle serait classée « moyenne » sans normalisation. Mais le réseau ne voit pas cette image en 1 920 pixels : il la redimensionne à 640. Le facteur d'échelle vaut $640/1\,920 = 0,333$, et l'aire vue par le réseau vaut $2\,700 \times 0,333^2 = 300$ pixels carrés. La voiture est donc, du point de vue du détecteur, un objet **petit**.

📁 **Chez nous.** Deux lignes de code portent toute cette logique :

```

212     # Facteur de reduction applique par le reseau a cette image.
213     echelle = TAILLE_REFERENCE / max(largeur, hauteur)
214     ...
215     aire = (reelles[i, 2] - reelles[i, 0]) * (reelles[i, 3] - reelles[i, 1])
216     aire *= echelle**2 # l'aire varie comme le CARRE de l'echelle

```

[scripts/evaluate.py:53-61, 213, 219-220]

Le ****2** est le point que le jury peut tester : une longueur est divisée par 3, donc une aire est divisée par 9.

⚠️ Vérification dans le dépôt

Le dépôt documente lui-même l'erreur commise et corrigée : « sans cette normalisation, des photos de plusieurs milliers de pixels de large rangent la totalité des véhicules dans la catégorie grand et l'analyse ne mesure plus rien. Ce point est vérifié : c'est exactement ce qui s'est produit lors du premier essai. » [README.md:209-213 ; article/sections/methode.tex:156-163] Raconter cette erreur en soutenance est un point fort, pas une faiblesse.

⚠️ **Piège et question de jury.** Comparer nos chiffres par taille à ceux d'un article qui n'aurait pas normalisé : ils ne mesurent pas la même chose. **Question de jury** : « vos seuils de 32 et 96 pixels sont-ils arbitraires ? » Réponse : ce sont les seuils officiels du protocole COCO, que nous reprenons

pour être comparables à la littérature, et nous ajoutons la normalisation à 640 pixels sans laquelle ces seuils n'auraient aucun sens sur des images de vidéoprotection en haute résolution.

À retenir

Six métriques globales et une mesure ciblée. Les globales disent « le modèle est meilleur ». La mesure ciblée, le rappel par taille, dit « il est meilleur *là où le sujet le demandait* ». C'est cette dernière ligne qui répond à la question de recherche.

6 Nos résultats

Cette section commente *chaque ligne* du fichier `results/comparaison.md`, puis explique l'effet observé, puis énonce ce que ces chiffres ne prouvent pas.

6.1 Le cadre de la mesure

Élément	Valeur
Split évalué	test, 300 images, 2 551 boîtes de référence
Modèle de référence	<code>models/yolov8n.pt</code> (80 classes COCO)
Modèle fine-tuné	<code>models/finetuned/yolov8n_benin.pt</code> (4 classes)
Seuil de confiance	0,001 pour les métriques globales, 0,25 pour le rappel par taille et le débit
Seuil d'IoU	0,6 pour la validation Ultralytics, 0,5 pour l'appariement du rappel par taille

[results/comparaison.md:3-5 ; scripts/evaluate.py:43-44, 121-122]

6.2 Ligne par ligne

6.2.1 Ligne 1 : mAP@0,5 passe de 0,4380 à 0,8252, soit +88,4 %

Le modèle générique obtient une note de 0,44 sur 1 quand on lui demande simplement de recouvrir à moitié les bons véhicules. Après fine-tuning, 0,83. En français courant : **le modèle a presque doublé sa qualité de détection sur des critères permissifs**. C'est la ligne à citer quand on veut une phrase simple.

6.2.2 Ligne 2 : mAP@0,5:0,95 passe de 0,2959 à 0,6443, soit +117,8 %

C'est la métrique la plus exigeante, celle qui demande un rectangle bien placé et pas seulement au bon endroit approximatif. Elle progresse *davantage* que la précédente (+117,8 contre +88,4 pour cent), ce qui signifie que le fine-tuning n'a pas seulement appris à *voir* plus de véhicules, il a aussi appris à les *cadrer* plus finement. C'est cohérent avec la baisse continue de la perte de boîte et de la perte DFL pendant l'entraînement (section 4).

6.2.3 Ligne 3 : précision de 0,4959 à 0,8327, soit +67,9 %

Avant : une annonce sur deux est fausse. Après : plus de huit sur dix sont justes. Une partie de ce gain vient d'un effet mécanique qu'il faut savoir nommer : le modèle fine-tuné ne connaît que 4 classes, donc il ne peut plus produire de détection hors périmètre. Le protocole neutralise en grande partie cet avantage puisque le modèle COCO est restreint aux quatre indices véhicules lors de l'inférence [scripts/evaluate.py:123], mais la restriction ne supprime pas tout : le modèle COCO garde 80 sorties de classification, il répartit donc sa confiance différemment.

6.2.4 Ligne 4 : rappel de 0,4172 à 0,7138, soit +71,1 %

Le modèle générique rate près de six véhicules sur dix, le fine-tuné un peu moins de trois sur dix. C'est la ligne la plus parlante pour un non-technicien, parce que « rater un véhicule » se comprend immédiatement.

6.2.5 Ligne 5 : F1-score de 0,4532 à 0,7687, soit +69,6 %

Aucune information nouvelle : c'est la combinaison des deux lignes précédentes. Sa vertu est de résister à l'objection « vous avez amélioré l'un au détriment de l'autre » : ici les deux progressent, donc leur moyenne harmonique aussi.

6.2.6 Ligne 6 : débit de 37,2 à 43,2 images par seconde, soit +16,2 %

⚠ Vérification dans le dépôt

C'est la ligne la plus fragile du tableau, et il faut l'annoncer avant qu'on vous la conteste. L'architecture des deux modèles est identique : un gain de débit ne peut donc pas venir du réseau. Les deux explications avancées par l'article sont un post-traitement allégé (4 classes au lieu de 80 à trier lors de la suppression des non-maxima) et la variabilité de mesure d'un environnement partagé comme Google Colab. Aucune expérience n'a été conçue pour départager les deux. [article/sections/discussion.tex:47-53] Une troisième explication, mesurable dans le dépôt, s'ajoute : le modèle fine-tuné a 145 572 paramètres de moins que le modèle de référence, puisque sa tête produit 4 scores de classe au lieu de 80. **[fait et vérifié]**

Position à tenir : « ce n'est pas un résultat que nous revendiquons, c'est une observation que nous rapportons avec ses réserves. Le message défendable est que le fine-tuning ne coûte rien en débit. »

6.2.7 Lignes 7 à 9 : le rappel par taille, le cœur du sujet

Taille	Objets	Trouvés avant	Trouvés après	Rappel avant	Rappel après
petit	804	98	558	0,1219	0,6940
moyen	1 415	528	1 244	0,3731	0,8792
grand	332	263	316	0,7922	0,9518
Total	2 551	889	2 118	0,3485	0,8303

[results/comparaison.json:13-28, 42-57] **[fait et vérifié]** La ligne Total est une somme que nous avons calculée à partir des trois autres ; elle ne figure pas telle quelle dans le dépôt.

Traduction en objets réels : sur les mêmes 300 images, le modèle générique trouve 889 véhicules, le modèle fine-tuné en trouve 2 118. **1 229 véhicules de plus**, dont 460 petits, 716 moyens et 53 grands.

💡 L'idée en une phrase

La phrase à retenir mot pour mot : « le modèle pré-entraîné ne retrouvait qu'un petit véhicule sur huit ; le modèle fine-tuné en retrouve plus des deux tiers. »
[article/sections/discussion.tex:9-11]

6.3 Ce que « petit » veut dire concrètement

⚠ Vérification dans le dépôt

[fait et vérifié] Vérification faite pour ce document, directement sur les fichiers du dépôt. Les 300 images de test mesurent 1 920 par 1 080 pixels (299 sur 300 ; la dernière fait 1 470 par 1 080). Le facteur de normalisation vaut donc $640/1\,920 = 1/3$, et les aires sont divisées par 9.

Conséquence très concrète, et c'est la meilleure façon de faire comprendre le sujet à un jury :

Catégorie	Seuil après normalisation	Ce que cela vaut dans l'image d'origine
petit	moins de 1 024 pixels carrés	moins d'environ 96 par 96 pixels sur une image Full HD
moyen	de 1 024 à 9 216	de 96 par 96 à environ 288 par 288
grand	9 216 et plus	plus de 288 par 288 pixels

Autrement dit, un « petit » objet est un véhicule qui occupe moins de un deux-centième de la surface de l'image. Sur une photo d'embouteillage, ce sont tous les véhicules du second plan et au-delà.

6.4 Qui sont les petits objets ? Une vérification supplémentaire

⚠ Vérification dans le dépôt

[fait et vérifié] Calcul refait pour ce document sur les 300 images de test, avec exactement la même règle de normalisation que `scripts/evaluate.py`. Les totaux retrouvés sont 804, 1 415 et 332, identiques au dépôt au chiffre près, ce qui valide la reproduction. La ventilation par classe, elle, **ne figure nulle part dans le dépôt** : c'est un apport de ce document.

Taille	voiture	moto	bus	camion	Total
petit	152	600	24	28	804
moyen	452	744	93	126	1 415
grand	138	36	72	86	332
Total	742	1 380	189	240	2 551

💡 L'idée en une phrase

74,6 pour cent des petits objets sont des motos (600 sur 804). Le verrou « petits objets » et le verrou « deux-roues » sont donc, dans ce jeu, le même verrou. Cela renforce l'argument de similarité avec le trafic béninois : c'est exactement sur la catégorie des zémidjans que le gain se concentre.

6.5 Pourquoi l'écart explose sur les petits objets

Cinq explications, de la plus solide à la plus hypothétique. Les trois premières sont vérifiables dans le dépôt, les deux dernières sont des hypothèses que l'article assume comme telles.

1. **Un effet arithmétique de départ.** Un rappel qui passe de 0,1219 à 0,6940 progresse de 469 % ; un rappel qui passe de 0,7922 à 0,9518 ne peut mathématiquement pas progresser de plus de 26 %, puisque le maximum est 1. **Le pourcentage relatif est mécaniquement amplifié par un point de départ bas.** C'est la première chose à dire soi-même.
2. **Le modèle générique était presque aveugle sur cette catégorie.** Retrouver 98 objets sur 804 n'est pas « moins bon », c'est inutilisable en exploitation. Le fine-tuning fait passer la catégorie de l'inutilisable à l'exploitable, ce qui est un changement de nature plus qu'un changement de degré.
3. **L'entraînement fabrique délibérément des petits objets.** `mosaic: 1.0` assemble quatre images en une, donc divise environ par quatre la surface de chaque véhicule, et `scale: 0.7` autorise un zoom arrière important. [`configs/entrainement.yaml:28-29`]

4. **Le domaine d'origine ne contient pas ces configurations.** COCO est fait de photographies du quotidien, où un véhicule est rarement un rectangle de 40 pixels vu en plongée depuis un poteau. L'écart de domaine publié par les auteurs de BMD-45, un facteur 2,5 de mAP, décrit le même phénomène. [article/sections/discussion.tex:22-33]
5. **La densité du jeu d'entraînement.** 8,5 véhicules par image en moyenne apprennent au modèle à s'attendre à des scènes chargées, là où COCO contient peu d'objets par image. [article/sections/discussion.tex:16-20]

6.6 Ce que ces chiffres ne prouvent pas

C'est la sous-section la plus importante de tout le document. Un jury technique valorise davantage une limite énoncée par vous qu'un chiffre élevé.

6.6.1 Objection 1 : le +469 % est-il honnête ?

Faisons le test le plus dur possible, celui qu'un examinateur aguerri peut poser. Au lieu du pourcentage relatif, regardons la **part de la marge disponible** que chaque catégorie a récupérée. La marge disponible, c'est $1 - \text{rappel initial}$.

Taille	Gain relatif	Gain en points	Marge disponible	Part récupérée
petit	+469,4 %	+57,2	87,8	65,1 %
moyen	+135,6 %	+50,6	62,7	80,7 %
grand	+20,2 %	+16,0	20,8	76,8 %

Colonnes 3, 4 et 5 calculées pour ce document à partir des valeurs du dépôt ; elles ne figurent pas dans les fichiers de résultats.

⚠ Vérification dans le dépôt

Lecture honnête de ce tableau. Selon la mesure choisie, la conclusion change de nuance :

- en **pourcentage relatif**, le gain est d'autant plus fort que l'objet est petit : c'est ce que dit l'article, et c'est exact ;
- en **points de rappel**, l'ordre est le même : +57,2 pour les petits, +50,6 pour les moyens, +16,0 pour les grands ;
- en **part de la marge disponible**, les petits objets sont ceux qui en récupèrent le *moins* : 65 % contre 81 % pour les moyens.

Autrement dit, les petits objets restent la catégorie la plus difficile même après fine-tuning. Cela ne contredit pas la conclusion du projet, cela la précise.

La réponse à donner : « le verrou visé a été levé, pas supprimé. Nous passons d'un rappel de 0,12, inexploitable, à 0,69, exploitable. Les petits objets restent néanmoins la catégorie la plus difficile, et c'est pourquoi nous identifions la résolution d'entrée, l'inférence par tuiles et le placement des caméras comme les pistes suivantes, plutôt que le seul entraînement. » [article/sections/discussion.tex:38-45]

6.6.2 Objection 2 : avez-vous adapté le modèle au trafic béninois ?

Non, et c'est écrit dans le dépôt. Le jeu est indien, filmé à Bengaluru. Le rapprochement repose sur trois traits structurels : caméras CCTV fixes, forte densité, dominance des deux-roues à 56 %. [article/sections/discussion.tex:57-66] La formulation à éviter et celle à employer sont d'ailleurs écrites noir sur blanc dans la procédure du projet :

À éviter : « nous avons adapté YOLOv8 au trafic béninois ».

À dire : « faute de jeu de données béninois public, nous avons travaillé sur du trafic urbain dense d'un contexte comparable. » [docs/PROCEDURE.md:43-48]

6.6.3 Objection 3 : la comparaison est-elle équitable ?

C'est la question la plus technique, et la réponse est le point fort du projet. Trois précautions :

1. Les étiquettes du test sont réindexées vers l'espace de classes du modèle évalué, à la volée, sans modifier le jeu d'origine. [scripts/evaluate.py:73-108]
2. Le modèle COCO est restreint aux classes 2, 3, 5 et 7 lors de l'inférence, pour ne pas être pénalisé par des piétons ou des feux comptés en faux positifs. [scripts/evaluate.py:123, 186]
3. Les deux modèles passent par *la même fonction* de mesure, appelée deux fois depuis le même script. [scripts/compare_models.py:19, 104-106]

6.6.4 Objection 4 : et la contamination entre splits ?

Voir la section 2. Résumé : risque réel, documenté par nous, qui flatte les *deux* modèles de la même manière et n'invalide donc pas la comparaison, mais qui invite à lire les valeurs absolues comme des ordres de grandeur. [article/sections/discussion.tex:89-95]

6.6.5 Objection 5 : un seul entraînement, un seul jeu de test

C'est la limite la moins défendable et il faut la reconnaître sans détour. Un seul run, une seule graine, un seul découpage. Nous ne rapportons donc aucun intervalle de confiance et nous ne pouvons pas distinguer un vrai effet d'une variation de tirage. **absent du dépôt** pour toute mesure de dispersion, tout écart-type et toute répétition d'expérience. La procédure du projet recommandait d'ailleurs plusieurs entraînements [docs/PROCEDURE.md:385-387], ce qui n'a pas été fait faute de temps machine. **[pas fait]**

6.7 Biais et limites, la liste complète

Limite	Description	Statut
Domaine	Jeu indien, pas béninois ; aucun jeu béninois public n'existe	assumée et écrite
Échelle	3 000 images sur 45 000, modèle nano, 50 epochs	assumée et écrite
Convergence	Entraînement arrêté avant convergence, chiffres = plancher	assumée et écrite
Classes minoritaires	Bus 189 et camion 240 en test, pas de métrique par classe	assumée et écrite
Contamination	Caméras fixes, images voisines possibles entre splits	assumée et écrite
Débit	Gain de 16 pour cent non expliqué, matériel non enregistré	partiellement assumée
Répétitions	Un seul run, aucune mesure de dispersion	[pas fait]
Ablation	Effet de mosaic et scale non isolé	[pas fait]
Vie réelle	Aucun test sur des images de Cotonou	[pas fait]

[article/sections/discussion.tex:55-104]

Devant le jury

Si on vous attaque sur les limites, la posture gagnante est de les avoir énoncées avant.
Phrase type : « nous avons neuf limites identifiées, dont trois que nous n'avons pas eu le temps

de traiter : la répétition des expériences, l'étude d'ablation et le test sur images béninoises. Les six autres sont documentées dans l'article et n'invalident pas la comparaison, qui est notre résultat. »

À retenir

Le résultat du projet tient en une ligne et une réserve. La ligne : 1 229 véhicules détectés en plus sur les mêmes 300 images, et le gain est concentré sur les petits objets, dont 75 pour cent sont des motos. La réserve : ce résultat est établi sur un domaine comparable, pas sur le domaine cible.

7 Le code de bout en bout

7.1 La carte du dépôt

Fichier	Rôle en une phrase	Écrit par
configs/import.yaml	où est le jeu téléchargé, et la table de correspondance des classes	vous
configs/dataset.yaml	où sont les images converties, et les 4 classes	le script d'import
configs/entrainement.yaml	tous les hyperparamètres du fine-tuning	vous
scripts/download_bmd45_subset.py	extraît un sous-ensemble de BMD-45 depuis Hugging Face	
scripts/import_dataset.py	convertit le jeu vers nos 4 classes et garantit un test isolé	
scripts/train_yolo.py	lance le fine-tuning	
scripts/evaluate.py	mesure un modèle, y compris le rappel par taille	
scripts/compare_models.py	mesure les deux modèles et produit le tableau	
scripts/detect_image.py	annote une image ou un dossier, pour les visuels	
scripts/tracer_courbes.py	retrace les courbes d'entraînement en PDF vectoriel	
notebooks/colab_entrainement.ipynb	déroule toute la chaîne sur un GPU gratuit	

⚠ Vérification dans le dépôt

Le tableau des scripts de docs/STRUCTURE.md:97-103 n'en liste que **cinq**, et l'arborescence de README.md:72-77 également : les deux fichiers `download_bmd45_subset.py` et `tracer_courbes.py` ont été ajoutés plus tard sans mise à jour de ces deux tableaux. Ils sont bien mentionnés ailleurs [README.md:113-119 ; article/figures/LISEZMOI.md:12-19]. Si un membre du jury lit le README, il comptera cinq scripts et en verra sept dans le dossier : autant le savoir.

7.2 Le flux général

data/raw → data/dataset → models/finetuned → results
 (jeu téléchargé) (converti) (modèle entraîné) (chiffres)

[docs/STRUCTURE.md:30-35]

7.3 Préalable : la ligne de commande sous Windows PowerShell

Toutes les commandes se lancent depuis la racine du projet. Le chemin du projet contient des espaces et un accent, ce qui impose deux précautions :

```
1 | cd "C:\Users\Andréa Afouda\Documents\Projets python et autres\Mon Projet ML\AMA\Projets_AMA\Projet1"
```

Les guillemets sont obligatoires à cause des espaces. Ensuite, deux façons de travailler :

```

1 | .\.\venv\Scripts\Activate.ps1
    puis python scripts/..., ou bien, plus sûr et sans activation :
1 | .venv\Scripts\python.exe scripts\evaluate.py --model models\yolov8n.pt --split test

```

⚠ Vérification dans le dépôt

Trois pièges propres à PowerShell, à connaître si vous faites une démonstration en direct. **Un** : l'activation peut être refusée avec le message *execution of scripts is disabled on this system*. Solution ponctuelle : `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`. **Deux** : l'enchaînement `commande1 && commande2` n'existe pas dans Windows PowerShell 5.1 ; écrivez les commandes l'une après l'autre. **Trois** : les scripts acceptent indifféremment les barres obliques et les contre-obliques dans les chemins, car ils utilisent `pathlib`.

7.4 configs/import.yaml

Rôle. Deux informations, et rien d'autre : le dossier du jeu téléchargé, et la table de correspondance entre ses classes et les nôtres.

```

1 | source: data/raw/bmd45_subset # ligne 6
2 |
3 | correspondance: # ligne 16
4 |   Hatchback: voiture
5 |   Sedan: voiture
6 |   ...
7 |   Three-wheeler: ignorer
8 |   Bicycle: ignorer
9 |   Other: ignorer

```

[configs/import.yaml:6, 16-52]

Ce qui casse souvent. Une classe du jeu source absente de la liste : le script s'arrête et affiche la liste réelle des classes, qu'il suffit de recopier. C'est volontaire, voir la section 2.

⚠ Vérification dans le dépôt

La table contient des variantes en minuscules (`van`, `bus`, `truck`, `motorcycle`, `bike`, etc.) qui n'existent pas dans BMD-45, dont les 14 noms commencent par une majuscule. Ces lignes sont inoffensives : une entrée de la table qui ne correspond à aucune classe source est simplement ignorée. Elles servent de filet si l'on change de jeu de données. [configs/import.yaml:18-52 ; scripts/download_bmd45_subset.py:65-81]

7.5 configs/dataset.yaml

Rôle. Décrire le jeu converti à Ultralytics. **Il est régénéré à chaque import : ne l'écrivez jamais à la main.** [configs/dataset.yaml:3-4]

```

1 | path: C:/Users/.../Projet1/data/dataset # chemin ABSOLU, voir plus bas
2 | train: train/images
3 | val: val/images
4 | test: test/images
5 |
6 | names:
7 |   0: voiture
8 |   1: moto
9 |   2: bus
10 |   3: camion

```

[configs/dataset.yaml:19-28]

Le point technique à savoir expliquer. Le chemin est absolu, et le fichier explique pourquoi : Ultralytics résout un `path` relatif par rapport à son propre réglage global `datasets_dir`, et non par rapport à l'emplacement du fichier. Comme le fichier est régénéré sur chaque machine par l'import, un chemin absolu est sans risque. [configs/dataset.yaml:14-17]

Ce qui casse souvent. Récupérer le dépôt sur une autre machine et lancer l'entraînement sans refaire l'import : le chemin absolu pointe alors vers un dossier qui n'existe pas. Le remède tient en une commande, relancer `import_dataset.py`.

7.6 configs/entrainement.yaml

Détaillé ligne par ligne en section 4. Retenir seulement le principe : *aucun hyperparamètre ne doit être écrit en dur ailleurs*, afin qu'un run soit entièrement décrit par ce fichier.

[configs/entrainement.yaml:2]

7.7 scripts/download_bmd45_subset.py

Rôle. Télécharger un sous-ensemble de BMD-45 sans rapatrier les 45 000 images, et l'écrire au format YOLO.

Logique en quatre blocs.

1. **Lire les noms de classes** depuis les métadonnées Hugging Face, avec une liste de secours codée en dur pour BMD-45 si les métadonnées ne les exposent pas. [scripts/download_bmd45_subset.py:51-86]
2. **Ouvrir le jeu en flux et le mélanger** avec une graine, de sorte que l'ordre soit reproductible :

```
161 | ds = load_dataset(repo, split=hf_split, streaming=True) # rien n'est telecharge ici
162 | ds = ds.shuffle(seed=seed, buffer_size=shuffle_buffer) # melange reproductible
```

3. **Convertir chaque annotation** au format YOLO normalisé et écrire l'image en JPEG qualité 95. [scripts/download_bmd45_subset.py:98-111, 179-192]
4. **Écrire le data.yaml** du jeu téléchargé, sans clé `test` puisqu'il n'y en a pas. [scripts/download_bmd45_subset.py:201-215, 252]

Commande exacte.

```
1 | .venv\Scripts\python.exe scripts\download_bmd45_subset.py --train 2400 --val 600 --seed
  | 42
```

Ce qu'elle affiche. La liste des 14 classes détectées, puis une ligne de progression toutes les 200 images pour chaque split, enfin un récapitulatif du type « Train : 2400 images / Val : 600 images / Test : non exporté ». [scripts/download_bmd45_subset.py:224-258]

Ce qui casse souvent.

- **Coupure réseau.** Relancez la même commande : le script compte les paires image plus étiquette déjà complètes et reprend où il s'était arrêté. [scripts/download_bmd45_subset.py:114-133, 151-159]
- **Module datasets manquant.** Il figure dans `requirements.txt:5`.
- **Lenteur.** Le notebook Colab annonce environ 700 Mo, rapides depuis Colab, beaucoup plus lents depuis une connexion domestique. [notebooks/colab_entrainement.ipynb, cellule 0]

7.8 scripts/import_dataset.py

Rôle. Traduire le jeu téléchargé vers nos quatre classes, normaliser les noms de dossiers, garantir un jeu de test isolé, et régénérer `configs/dataset.yaml`.

Logique en six blocs.

1. **Lire les classes du jeu source** dans son `data.yaml`, en gérant les deux formes possibles, liste ou dictionnaire. [scripts/import_dataset.py:50-65]

2. **Construire la table de conversion** et refuser d'aller plus loin si une classe n'est pas traitée.

[scripts/import_dataset.py:78-116]

3. **Retrouver les dossiers de splits** malgré les noms variables selon les exportateurs, grâce à une table d'alias :

```
43 ALIAS_SPLITS = {
44     "train": ["train", "training"],
45     "val": ["val", "valid", "validation"],
46     "test": ["test", "testing"],
47 }
```

4. **Copier chaque split** en réécrivant les étiquettes. Détail d'efficacité : les images sont liées en dur plutôt que copiées, ce qui ne duplique aucun octet sur le disque, avec une copie classique en secours si le système ne le permet pas :

```
150     cible = images_dst / image.name
151     try:
152         os.link(image, cible) # lien dur : instantane, zero octet duplique
153     except OSError:
154         shutil.copy2(image, cible) # secours : copie classique
```

5. **Dériver le jeu de test** depuis la validation si le jeu source n'en fournit pas, avec la graine 42.

[scripts/import_dataset.py:166-190]

6. **Écrire configs/dataset.yaml** et afficher le tableau des effectifs par split et par classe.

[scripts/import_dataset.py:193-285]

Commande exacte.

```
1| .venv\Scripts\python.exe scripts\import_dataset.py
```

Ce qu'elle affiche. D'abord la liste des 14 classes avec leur destination (Hatchback -> voiture, Bicycle -> ignoree (choix explicite)), puis le message signalant que le test a été prélevé sur la validation, puis le tableau des effectifs, celui de la section 2, enfin un avertissement rappelant qu'une classe à moins de quelques dizaines d'instances en test ne donnera pas de métrique fiable.

Ce qui casse souvent.

- **Une classe non traitée** : arrêt volontaire, message explicite.
- **Le script efface data/dataset/** avant de le reconstruire. Tout ce que vous auriez ajouté à la main dans ce dossier est perdu. [scripts/import_dataset.py:136-139 ; docs/STRUCTURE.md:71-73]
- **Relancer l'import après l'entraînement** redécoupe le test avec la même graine, donc à l'identique ; mais si vous changez la graine, vos anciens résultats deviennent incomparables aux nouveaux.

7.9 scripts/train_yolo.py

Rôle. Lancer le fine-tuning, en ne lisant que le fichier de configuration, puis copier les meilleurs poids à un emplacement stable.

Logique en quatre blocs.

1. **Charger la configuration** et vérifier que le jeu existe, avec un message qui dit quoi faire :

```
42     if not donnees.exists():
43         sys.exit(f"{donnees} introuvable.\nLancez d'abord : python scripts/import_dataset.py")
```

2. **Charger les poids de départ** et lancer l'entraînement, en passant les neuf réglages d'augmentation par dépliage de dictionnaire. [scripts/train_yolo.py:52-66]
3. **Retrouver le meilleur instantané**, que la bibliothèque nomme `best.pt` dans le dossier du run.
4. **Le copier** sous un nom stable, pour que les scripts suivants aient un chemin à viser :

```

71     meilleur = Path(resultats.save_dir) / "weights" / "best.pt"
72     if meilleur.exists():
73         cible = DESTINATION_POIDS / f"{config['nom_run']}.pt"
74         shutil.copy2(meilleur, cible)

```

[scripts/train_yolo.py:68-78]



L'idée en une phrase

best.pt n'est pas le modèle de la dernière epoch, c'est celui de la *meilleure* epoch sur le jeu de validation. C'est précisément pour cela qu'il faut un jeu de test distinct : la validation a déjà servi à choisir.

Commande exacte.

```
1| .venv\Scripts\python.exe scripts\train_yolo.py
```

et, pour forcer la carte graphique :

```
1| .venv\Scripts\python.exe scripts\train_yolo.py --device 0
```

Ce qu'elle affiche. Les deux chemins de départ, puis le tableau d'Ultralytics epoch par epoch avec les trois pertes et les quatre métriques, puis le chemin des poids copiés et la commande suivante à lancer.

Ce qui casse souvent.

- **Durée.** Plusieurs heures sur processeur, environ une heure sur GPU T4, mesurée à 54,9 minutes pour notre run. [docs/PROCEDURE.md:250-252]
- **Mémoire insuffisante.** Réduisez **batch** avant de réduire **taille_image** : c'est la résolution qui conditionne la détection des petits objets. [docs/PROCEDURE.md:273-275]
- **Relancer avec le même nom_run.** Le paramètre **exist_ok=False** [scripts/train_yolo.py:64] empêche d'écraser le dossier du run existant ; en revanche le fichier **models/finetuned/yolov8n_benin.pt**, lui, sera écrasé car son nom vient de **nom_run**. Sauvegardez-le avant toute nouvelle tentative si vous tenez à vos chiffres actuels.

7.10 scripts/evaluate.py, le cœur méthodologique

Rôle. Mesurer un modèle, quel que soit son espace de classes, et produire le rappel ventilé par taille d'objet.

Logique en cinq blocs.

1. **Détecter l'espace de classes du modèle.** Une règle simple et lisible :

```

68 def modele_est_coco(modele: YOLO) -> bool:
69     """Un modele pre-entraine COCO expose 80 classes ; le notre en expose 4."""
70     return len(modele.names) >= 80

```

2. **Fabriquer une vue réindexée du jeu de test** si le modèle est un modèle COCO. Les images sont liées en dur, seules les étiquettes sont réécrites, et la vue est supprimée après usage. [scripts/evaluate.py:73-108, 295-296]
3. **Calculer les métriques globales** via le validateur de la bibliothèque, avec les seuils voulus. [scripts/evaluate.py:115-137]
4. **Calculer le rappel par taille et le débit**, à la main, sur les mêmes images. [scripts/evaluate.py:177-247]
5. **Écrire un rapport JSON** et l'afficher lisiblement. [scripts/evaluate.py:310-341]

💡 L'idée en une phrase

Le bloc 2 mérite un mot de plus, car c'est là que se joue l'équité de la comparaison. Pour évaluer le modèle COCO, on ne modifie ni le jeu de test ni le modèle : on crée une *vue* temporaire du jeu, où 0 est remplacé par 2, 1 par 3, et ainsi de suite. Un fichier YAML jetable est écrit à côté, avec les trois clés `train`, `val` et `test` pointant toutes sur cette vue, parce qu'Ultralytics les exige toutes les trois même quand on n'en valide qu'une. [scripts/evaluate.py:270-296]

Commande exacte.

```
1 | .venv\Scripts\python.exe scripts\evaluate.py --model models\yolov8n.pt --split test
```

Ce qu'elle affiche. Une ligne d'identification du modèle (« 80 classes, indexation COCO »), les barres de progression du validateur, puis un bloc de résultats :

```
1 | --- models/yolov8n.pt (split test) ---
2 | mAP@0.5 : 0.4380
3 | mAP@0.5:0.95 : 0.2959
4 | Precision : 0.4959
5 | Recall : 0.4172
6 | F1-score : 0.4532
7 | Debit : 37.2 FPS
8 |
9 | Rappel par taille d'objet (convention COCO) :
10 | petit 98/804 = 0.1219
11 | moyen 528/1415 = 0.3731
12 | grand 263/332 = 0.7922
```

Reconstitué à partir du format d'affichage [scripts/evaluate.py:310-321] et des valeurs réelles [results/comparaison.json:5-28].

⚠ Vérification dans le dépôt

Le fichier écrit par cette commande s'appellerait `results/eval_yolov8n_test.json` [scripts/evaluate.py:338]. Ce fichier **absent du dépôt** : le dossier `results/` ne contient que les trois `comparaison.*` et le dossier `entrainements/`. Les chiffres publiés viennent donc tous de `compare_models.py`, qui appelle la même fonction d'évaluation. Aucune conséquence sur la validité, mais ne dites pas « nous avons aussi les rapports individuels » : ils ne sont pas dans le dépôt.

Ce qui casse souvent.

- **Jeu absent** : message explicite renvoyant vers l'import. [scripts/evaluate.py:259-263]
- **Chemin absolu périmé** dans `configs/dataset.yaml` après un changement de machine.
- **Dossier `_vue_coco_test` resté en place** après une interruption. Il est ignoré par Git [.gitignore:46-47] et sera écrasé au prochain lancement. [scripts/evaluate.py:84-87]

7.11 scripts/compare_models.py, le livrable

Rôle. Évaluer les deux modèles et écrire le tableau comparatif dans trois formats.

Logique en trois blocs.

1. **Réutiliser la fonction d'évaluation**, sans la réécrire. C'est la garantie que les deux modèles sont mesurés par le même code :

```
19 | from evaluate import CONFIG_DATASET, DOSSIER_RESULTATS, RACINE, evaluer
20 | ...
21 | base = evaluer(base_path, args.data, args.split, args.device) # modele COCO
22 | finetune = evaluer(fine_path, args.data, args.split, args.device) # notre modele
```


[scripts/compare_models.py:19, 104-106]

2. **Construire les lignes** et calculer l'écart relatif, avec une protection contre la division par zéro :

```
30 def ecart_relatif(avant: float, apres: float) -> str:
31     if avant == 0:
32         return "n/a" if apres == 0 else "+inf"
33     return f"{{(apres - avant) / avant * 100:+.1f}} %"
```

3. **Écrire** `comparaison.md`, `comparaison.csv` et `comparaison.json`.
[scripts/compare_models.py:116-123]

Vérifions un écart à la main, pour être capable de le refaire au tableau : $(0,6940 - 0,1219) / 0,1219 \times 100 = 469,4 \%$. **[fait et vérifié]**

Commande exacte.

```
1 | .venv\Scripts\python.exe scripts\compare_models.py
```

Ce qu'elle affiche. Les deux blocs d'évaluation successifs, puis le tableau aligné en colonnes, puis la ligne « Écrit dans results/ : comparaison.md, comparaison.csv, comparaison.json ».

Ce qui casse souvent.

- **Lancer le script depuis un autre dossier.** La ligne `from evaluate import ...` fonctionne parce que Python ajoute automatiquement le dossier du script exécuté au chemin de recherche. Un `import` en tant que module, depuis un autre programme, échouerait.
- **Poids fine-tunés absents :** le script s'arrête sur `Modele introuvable`. [scripts/evaluate.py:253-254]
- **Durée :** de 20 à 40 minutes, puisque chaque modèle est passé deux fois sur les images, une fois par le validateur et une fois pour le rappel par taille. [docs/PROCEDURE.md:313]

7.12 scripts/detect_image.py

Rôle. Produire les visuels qualitatifs : la même scène vue par les deux modèles.

Le bloc à connaître est le filtrage des classes, qui reprend la même logique que l'évaluation :

```
57 # Le modele pre-entraîne connaît 80 classes : on ne garde que les vehicules.
58 # Le modele fine-tune n'en connaît que 4, toutes pertinentes.
59 classes = COCO_VEHICULES if len(modele.names) >= 80 else None
```

[scripts/detect_image.py:57-59, 23]

Le fichier de sortie est suffixé du nom du modèle, ce qui permet de placer les deux images côte à côte sans les confondre. [scripts/detect_image.py:63]

Commandes exactes.

```
1 | .venv\Scripts\python.exe scripts\detect_image.py data\dataset\test\images\val_000556.jpg
  | --model models\yolov8n.pt

1 | .venv\Scripts\python.exe scripts\detect_image.py data\dataset\test\images\val_000556.jpg
  | --model models\finetuned\yolov8n_benin.pt
```

Ce qu'elle affiche. Une ligne par image : le nom, le nombre de véhicules détectés, le nom du fichier écrit dans `data/outputs/`. [scripts/detect_image.py:65]

Ce qui casse souvent. Passer un dossier au lieu d'un fichier traite toutes les images, ce qui peut être long. Et surtout : pensez à flouter plaques et visages sur toute capture destinée aux diapositives.

[README.md:244-245]

7.13 scripts/tracer_courbes.py

Rôle. Retracer les courbes d'entraînement en PDF vectoriel, parce que le `results.png` produit par la bibliothèque devient illisible une fois réduit à la largeur d'une colonne d'article.

[scripts/tracer_courbes.py:1-10]

Deux détails techniques défendables.

```
23| matplotlib.use("Agg") # pas d'affichage interactif : on écrit un fichier
```

et, dans la lecture du CSV, le nettoyage des en-têtes, sans lequel les colonnes ne seraient jamais trouvées :

```
63|     for cle in lignes[0]:
64|         propre = cle.strip() # Ultralytics aligne les en-tetes avec des espaces
65|         colonnes[propre] = [float(ligne[cle]) for ligne in lignes]
```

[scripts/tracer_courbes.py:23, 48-67]

Commande exacte.

```
1| .venv\Scripts\python.exe scripts\tracer_courbes.py
```

Ce qu'elle affiche. Une seule ligne :

```
1| Figure écrite : article\figures\courbes_entrainement.pdf
```

Ce qui casse souvent. L'absence de `results/entrainements/yolov8n_benin/results.csv`. Ce fichier est pourtant versionné par exception [.gitignore:37-44], donc présent dans toute copie du dépôt.

7.14 notebooks/colab_entrainement.ipynb

Rôle. Dérouler toute la chaîne sur un GPU gratuit, sans rien installer, et sauvegarder les résultats sur Google Drive.

Cellule	Ce qu'elle fait
1	!nvidia-smi : vérifier que le GPU est bien actif
2	cloner le dépôt GitHub dans <code>projet1</code>
3	installer seulement ce qui manque, sans réinstaller PyTorch que Colab fournit déjà
5	monter Google Drive, de façon facultative : en cas de refus, le notebook continue
6	restaurer une archive partielle depuis Drive si elle existe, sinon télécharger, puis archiver même partiellement
7	import_dataset.py
8	train_yolo.py --device 0
9	compare_models.py --device 0
10	afficher le tableau comparatif dans le notebook
11	copier poids et résultats sur Drive, à exécuter avant de fermer

[notebooks/colab_entrainement.ipynb]

Ce qui casse souvent.

- **Oublier d'activer le GPU** : l'entraînement passerait sur processeur, donc plusieurs heures.
- **Fermer la session sans exécuter la dernière cellule** : tout est perdu, une session Colab est éphémère.
- **La coupure en plein téléchargement** : la cellule 6 archive même un téléchargement partiel sur Drive, précisément pour ne pas repartir de zéro.

⚠ Vérification dans le dépôt

Deux observations de traçabilité sur le run qui a produit nos chiffres. **Un** : le fichier `args.yaml` du run indique des chemins en `/content/projet1/projet1/...` [results/entrainements/yolov8n_benin/args.yaml:3, 15, 116], alors que le notebook clone dans `/content/projet1`. Le run vient donc d'une session où le dépôt s'est retrouvé dans un sous-dossier supplémentaire. Sans conséquence sur les chiffres, mais autant le savoir si la question

est posée. **Deux** : `models/yolov8n.pt` n'est pas versionné `[.gitignore:15]`, donc absent d'un clone frais ; c'est Ultralytics qui le retélécharge automatiquement au premier appel, comme l'annonce le README `[README.md:104-105]`.

7.15 Le chemin complet, en six commandes

```
1 .venv\Scripts\python.exe scripts\download_bmd45_subset.py --train 2400 --val 600 --seed 42
2 .venv\Scripts\python.exe scripts\import_dataset.py
3 .venv\Scripts\python.exe scripts\evaluate.py --model models\yolov8n.pt --split test
4 .venv\Scripts\python.exe scripts\train_yolo.py
5 .venv\Scripts\python.exe scripts\compare_models.py
6 .venv\Scripts\python.exe scripts\detect_image.py data\dataset\test\images\val_000556.jpg
```

`[docs/PROCEDURE.md:408-414, adapté à PowerShell]`

✦ À retenir

Trois principes de conception à savoir citer, parce qu'ils répondent à des questions de jury probables. **Un** : aucun seuil ni hyperparamètre codé en dur, tout vit dans `configs/`. **Deux** : `compare_models.py` importe `evaluate.py` au lieu de le dupliquer, donc les deux modèles passent par le même code de mesure. **Trois** : `import_dataset.py` refuse d'importer plutôt que de produire silencieusement des métriques fausses. `[docs/STRUCTURE.md:105-111]`

8 Les graphiques

Le dépôt contient cinq visuels. Pour chacun : ce qu'il montre, comment le lire axe par axe, à quoi ressemblerait un bon et un mauvais résultat, et ce que montre le nôtre.

8.1 La grille de 10 courbes d'Ultralytics

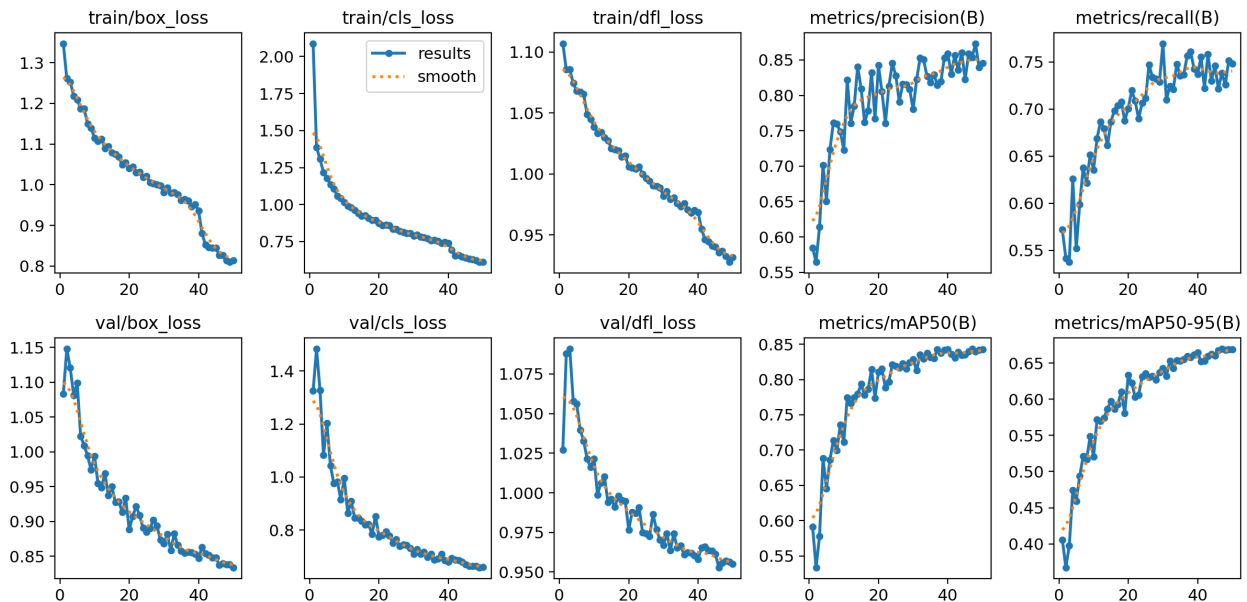


FIGURE 1 : `results/entrainements/yolov8n_benin/results.png`, produit automatiquement pendant l'entraînement.

Ce qu'il montre. Dix petites courbes, une par grandeur suivie, avec en abscisse le numéro d'epoch de 1 à 50. La ligne bleue est la valeur brute, la ligne orange en pointillés une version lissée.

Comment le lire, panneau par panneau.

Panneau	Grandeur	Sens attendu
1 à 3, en haut	pertes sur l'entraînement (boîte, classification, DFL)	descendre
4 et 5, en haut	précision et rappel sur la validation	monter
1 à 3, en bas	pertes sur la validation	descendre
4 et 5, en bas	mAP@0,5 et mAP@0,5:0,95 sur la validation	monter

À quoi ressemble un mauvais résultat. Trois signatures d'échec faciles à reconnaître : les pertes de validation qui remontent alors que celles d'entraînement descendent, c'est du surapprentissage ; des courbes plates dès le début, le modèle n'apprend rien, souvent un taux d'apprentissage mal réglé ; des courbes en dents de scie violentes qui ne se stabilisent jamais, souvent un lot trop petit ou un taux trop élevé.

Ce que montre le nôtre.

1. Les six courbes de perte descendent, en haut comme en bas. Aucune ne remonte durablement.
2. Les courbes de précision et de rappel oscillent beaucoup, ce qui est normal : elles sont mesurées à un seuil de confiance choisi automatiquement et sont donc plus instables que la mAP.
3. Les deux courbes de mAP montent vite jusqu'à l'epoch 20 environ, puis s'aplatissent sans jamais redescendre.

4. **Un décrochage net vers l'époch 41** sur les trois pertes d'entraînement, en haut. Ce n'est pas un progrès du modèle : c'est `close_mosaic: 10` qui désactive la mosaïque pour les dix dernières epochs, donc des images plus faciles. [results/entrainements/yolov8n_benin/args.yaml:27] **[fait et vérifié]**

Devant le jury

Si on vous montre cette figure et qu'on vous demande « que se passe-t-il à l'époch 41 ? », la bonne réponse est : la mosaïque est désactivée pour les dix dernières epochs, ce qui rend les images d'entraînement plus faciles et fait chuter les pertes d'entraînement ; les pertes de validation, elles, ne présentent pas de rupture équivalente, ce qui confirme qu'il s'agit d'un changement de difficulté et non d'un saut de performance.

8.2 Notre figure d'article, deux panneaux

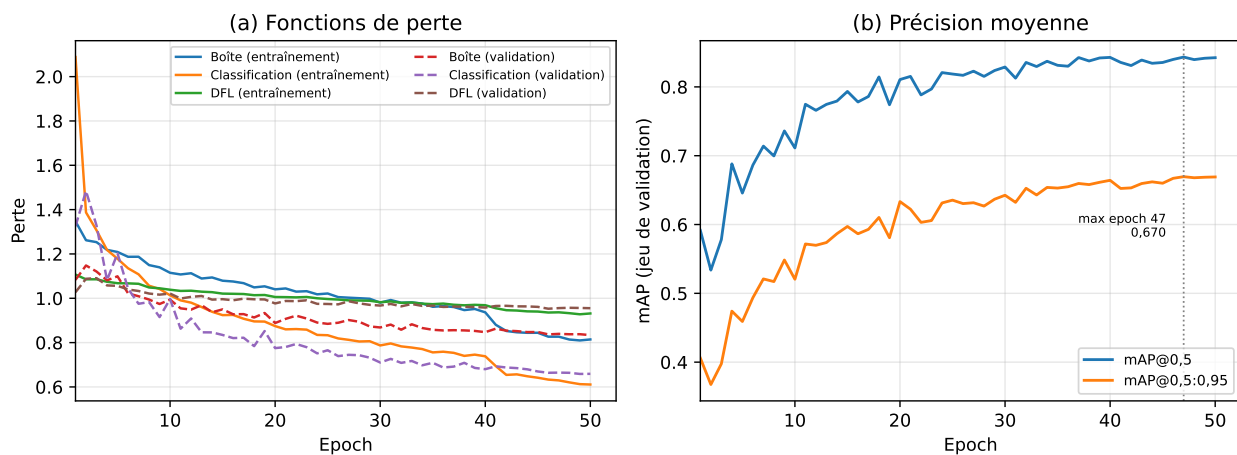


FIGURE 2 : article/figures/courbes_entrainement.pdf, produite par scripts/tracer_courbes.py à partir du même results.csv.

Ce qu'il montre. Les mêmes données, retracées en PDF vectoriel avec une police lisible et seulement les courbes commentées dans l'article. [scripts/tracer_courbes.py:1-10]

Comment le lire.

- **Panneau (a), les pertes.** Abscisse : l'époch, de 1 à 50. Ordonnée : la valeur de la perte, sans unité. Six courbes : trait plein pour l'entraînement, pointillés pour la validation. **Ce qu'il faut regarder : l'écart entre un trait plein et son pointillé de même couleur.** S'il se creuse, c'est du surapprentissage.
- **Panneau (b), la mAP.** Abscisse : l'époch. Ordonnée : la mAP sur le jeu de validation, entre 0 et 1. Deux courbes, mAP@0,5 en haut et mAP@0,5:0,95 en dessous, la seconde étant toujours inférieure puisqu'elle est plus exigeante. Une ligne verticale grise pointillée marque l'époch 47, maximum de la mAP@0,5:0,95, annoté avec sa valeur. [scripts/tracer_courbes.py:87-99]

Ce que montre le nôtre. Les traits pleins et les pointillés restent parallèles du début à la fin : pas de surapprentissage. La mAP@0,5:0,95 plafonne à 0,670 à l'époch 47 et vaut encore 0,669 à l'époch 50 : le modèle a atteint un plateau mais n'a pas commencé à se dégrader. [results/entrainements/yolov8n_benin/results.csv:48, 51] **[fait et vérifié]**

⚠ Vérification dans le dépôt

Pourquoi deux figures pour les mêmes données ? Parce que le `results.png` d'Ultralytics devient illisible une fois réduit à la largeur d'une colonne d'article et pixellisé à l'impression. La figure de l'article est régénérable par une commande, et son fichier source `results.csv` est versionné pour cette raison. [article/figures/LISEZMOI.md:12-30]

À noter : ce même fichier LISEZMOI affirme que « ce CSV n'est pas versionné (`.gitignore`) ». C'est **faux depuis** l'ajout d'une exception dans `.gitignore:37-44` : le fichier est bien suivi par Git. Une incohérence de documentation, sans conséquence technique.

8.3 La courbe précision-rappel

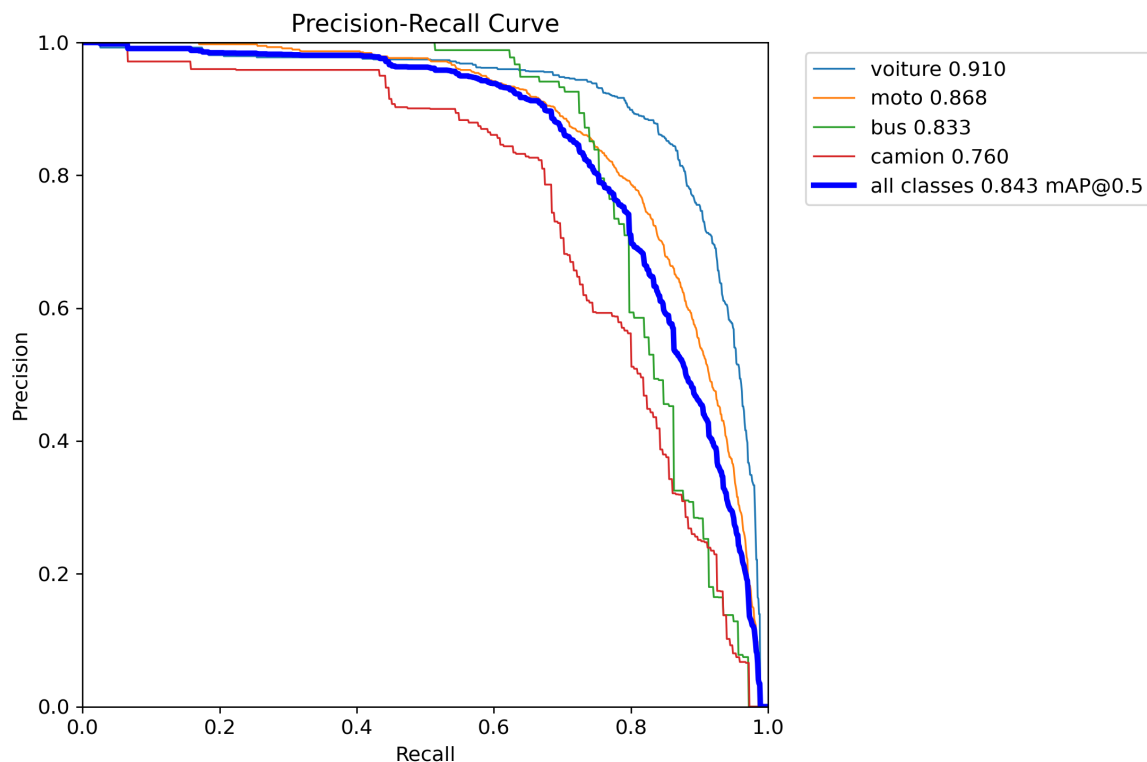


FIGURE 3 : `results/entrainements/yolov8n_benin/BoxPR_curve.png`.

Ce qu'il montre. Le comportement complet du modèle fine-tuné quand on fait varier le seuil de confiance, une courbe par classe plus une moyenne.

Comment le lire, axe par axe.

- **Abscisse, le rappel, de 0 à 1.** On se déplace vers la droite en *baissant* le seuil de confiance : le modèle devient plus permissif, trouve plus de véhicules.
- **Ordonnée, la précision, de 0 à 1.** Elle dit quelle proportion des détections est juste au seuil correspondant.
- **L'aire sous chaque courbe** est l'AP de la classe, indiquée dans la légende. La courbe bleue épaisse est la moyenne des quatre.

À quoi ressemble un bon et un mauvais résultat. Une courbe idéale resterait collée au plafond jusqu'à un rappel de 1, puis tomberait à pic : aire de 1. Une courbe mauvaise s'effondrerait dès un rappel de 0,2. Une courbe en escalier très marqué signale un effectif faible : chaque objet fait bouger la courbe visiblement.

Ce que montre le nôtre.

Classe	Aire sous la courbe
voiture	0,910
moto	0,868
bus	0,833
camion	0,760
toutes classes, mAP@0,5	0,843

[fait et vérifié]

Trois lectures. **Un** : les quatre courbes restent au-dessus de 0,9 de précision jusqu'à un rappel d'environ 0,6, ce qui est le comportement d'un modèle utilisable. **Deux** : la courbe du camion, en rouge, est la plus basse et la plus en escalier, ce qui est cohérent avec ses effectifs faibles. **Trois** : cette figure porte sur le jeu de **validation**, pas sur le jeu de test. Sa mAP@0,5 de 0,843 est à rapprocher du 0,8252 mesuré sur le test [results/comparaison.md:9]. L'écart est faible, à peine deux points, ce qui est un bon signe : le modèle choisi sur la validation se comporte presque aussi bien sur des images qu'il n'a jamais vues.

Devant le jury

Question probable : « pourquoi votre mAP de validation, 0,843, et votre mAP de test, 0,825, diffèrent-elles ? » **Réponse** : parce que le modèle conservé est celui qui obtenait le meilleur score sur la validation, donc la validation est légèrement optimiste par construction. Un écart de deux points est faible, et c'est précisément la raison pour laquelle nous publions les chiffres du test et non ceux de la validation.

8.4 La matrice de confusion normalisée

Ce qu'il montre. Ce que le modèle répond, pour chaque type de véhicule réellement présent.
Comment le lire, axe par axe.

- **Colonnes, en bas : la vérité.** Cinq colonnes : voiture, moto, bus, camion, et *background* pour « rien ».
- **Lignes, à gauche : la prédiction.** Mêmes cinq entrées.
- **Chaque colonne est normalisée**, donc ses valeurs se lisent comme des proportions de la vérité correspondante.
- **La diagonale** est la bonne réponse. Plus elle est foncée, mieux c'est.

À quoi ressemble un bon et un mauvais résultat. Une matrice idéale est une diagonale noire sur fond blanc. Une matrice mauvaise a des taches sombres hors diagonale, ou une colonne *background* chargée, signe de nombreuses détections sur du vide.

Ce que montre la nôtre.

Case	Valeur	Lecture
moto prédite moto	0,89	la classe majoritaire est très bien reconnue
voiture prédite voiture	0,72	correct
camion prédit camion	0,54	moitié seulement
bus prédit bus	0,49	la classe la plus faible
bus prédit camion	0,24 et 0,23	confusion réciproque entre les deux gros volumes
colonne <i>background</i>	0,53 vers moto	beaucoup de détections de motos sans annotation correspondante

[fait et vérifié]



FIGURE 4 : results/entrainements/yolov8n_benin/confusion_matrix_normalized.png.

⚠ Vérification dans le dépôt

Deux mises en garde à formuler vous-même. **Un** : cette matrice est calculée sur le jeu de **validation**, pendant l'entraînement, et non sur le jeu de test qui produit le tableau comparatif. Ne la présentez pas comme illustrant vos résultats publiés. **Deux** : elle est produite avec un seuil de confiance très bas, ce qui gonfle la colonne *background* : beaucoup de ces détections disparaîtraient au seuil d'exploitation de 0,25. La valeur de 0,53 vers la moto ne signifie donc pas que le modèle hallucine une moto sur deux.

💬 Devant le jury

Question redoutable : « votre modèle confond bus et camions, n'est-ce pas gênant pour de la verbalisation ? » **Réponse en trois phrases** : la confusion est sémantique, pas géométrique, le véhicule est bien détecté mais mal nommé. Notre mesure principale, le rappel par taille, est volontairement agnostique à la classe parce que la question posée est « le véhicule a-t-il été vu ». Pour un usage de verbalisation, la classe fine importerait davantage, et il faudrait alors rééquilibrer le jeu d'entraînement, où le bus ne compte que 1 298 instances contre 11 641 pour la moto.

8.5 Les deux images de démonstration

Ce qu'elles montrent. La même scène d'embouteillage, annotée par les deux modèles.

Comment les lire. Il n'y a pas d'axe : on compte les rectangles, et surtout on regarde *où* ils sont. À gauche, les quatre boîtes entourent les véhicules du premier plan, gros et bien séparés : le



FIGURE 5 : À gauche, YOLOv8n pré-entraîné : 4 véhicules détectés. À droite, YOLOv8n fine-tuné : 44 véhicules détectés. Même image du jeu de test (val_000556), même seuil de confiance 0,25. [article/figures/LISEZMOI.md:3-10]

camion frigorifique, deux voitures, un véhicule blanc. À droite, les rectangles couvrent aussi tout le peloton de deux-roues et la file de véhicules qui remonte vers le fond de la scène.

Ce que montre la nôtre, et pourquoi cette image est bien choisie. C'est exactement l'illustration visuelle du tableau chiffré : le modèle générique voit les gros objets proches, et ignore la masse des petits objets lointains, qui sont ici surtout des deux-roues. C'est le rappel des petits objets, 0,1219 contre 0,6940, rendu visible.

⚠ Vérification dans le dépôt

Deux précisions de rigueur sur ces images. **Un** : les visages sont floutés à la source dans BMD-45, et la seule plaque partiellement lisible a été floutée par le groupe. [article/figures/LISEZMOI.md:8-10] **Deux** : ces deux nombres, 4 et 44, sont un cas particulier spectaculaire, choisi précisément pour cela. Ils ne représentent pas l'écart moyen ; l'écart moyen est celui du tableau. Le dire avant qu'on vous le fasse remarquer vous grandit.

✳ À retenir

Cinq visuels, deux jeux différents. Le **results.png**, les courbes de l'article, la courbe précision-rappel et la matrice de confusion portent tous sur le jeu de **validation**. Seuls le tableau comparatif et les deux images de démonstration portent sur le jeu de **test**. Ne mélangez pas les deux à l'oral.

9 Fiches de survie

9.1 Glossaire de tous les termes définis dans ce document

Terme	Définition courte
Ablation (étude d')	réentraîner en désactivant un réglage à la fois, pour isoler sa contribution
AP	<i>Average Precision</i> , aire sous la courbe précision-rappel d'une classe
Arrêt anticipé	interrompre l'entraînement si aucune amélioration pendant un nombre donné d'epochs
Augmentation de données	transformer aléatoirement les images d'entraînement pour varier les présentations
<i>Background</i>	la case « rien » de la matrice de confusion
<i>Backbone</i> , dorsale	le premier bloc du réseau, qui extrait des motifs de plus en plus abstraits
Batch	nombre d'images traitées ensemble avant une mise à jour des paramètres
BMD-45	jeu de données de 45 000 images de caméras CCTV de Bengaluru, notre source
Boîte englobante	le plus petit rectangle horizontal contenant l'objet
Carte d'activation	l'image de sortie d'une couche de convolution
Classe	catégorie d'objet, désignée par un indice entier
Classification	dire ce qu'il y a sur l'image, sans dire où
<code>close_mosaic</code>	nombre d'epochs finales pendant lesquelles la mosaïque est désactivée
COCO	jeu de données généraliste de 80 classes, sur lequel le modèle de référence est entraîné
Convolution	faire glisser un petit filtre sur l'image et calculer une somme pondérée
Cou, <i>neck</i>	le bloc qui mélange les informations issues de plusieurs échelles
Déséquilibre	effectifs très différents d'une classe à l'autre
DFL	<i>Distribution Focal Loss</i> , perte qui traite la position d'un bord comme une distribution
Epoch	un passage complet sur tout le jeu d'entraînement
F1-score	moyenne harmonique de la précision et du rappel
Faux négatif (FN)	un véhicule réel que le modèle n'a pas trouvé
Faux positif (FP)	une détection qui ne correspond à aucun véhicule réel
Filtre, noyau	le petit tableau de nombres que la convolution fait glisser
Fine-tuning	poursuivre l'entraînement d'un modèle déjà entraîné, sur un jeu spécialisé
Format YOLO	un dossier <code>images</code> et un dossier <code>labels</code> , une ligne par objet, coordonnées normalisées
FPS	images traitées par seconde
Fuite de données	information du jeu de test présente indirectement dans l'entraînement
Graine, <i>seed</i>	nombre qui initialise l'aléatoire, garantit la reproductibilité
HSV	teinte, saturation, valeur ; les trois axes de variation de couleur de l'augmentation
Hyperparamètre	régler fixé avant l'entraînement, non appris
Inférence	faire tourner le modèle sur une image pour obtenir des prédictions
IoU	aire de l'intersection divisée par aire de l'union de deux boîtes
Lien dur	deux noms de fichier pointant vers les mêmes octets sur le disque
mAP	moyenne des AP sur toutes les classes
mAP@0,5	mAP avec une exigence de recouvrement de 0,5
mAP@0,5:0,95	moyenne des mAP à dix seuils de recouvrement, de 0,5 à 0,95
Matrice de confusion	tableau croisant vérité et prédiction
<i>Mosaic</i>	augmentation qui assemble quatre images en une seule
NMS	suppression des non-maxima, élimine les boîtes redondantes
Paramètre	nombre appris par le réseau pendant l'entraînement
Patience	nombre d'epochs sans progrès toléré avant l'arrêt anticipé
Perte, <i>loss</i>	mesure chiffrée de l'écart entre prédiction et annotation
Précision	part des détections annoncées qui sont justes
Rappel	part des véhicules réels effectivement trouvés
Rappel par taille	le rappel, ventilé en petit, moyen et grand selon la convention COCO

9.2 Script oral de 5 minutes

Environ 700 mots, à dire posément. Les indications entre crochets sont des repères, pas du texte à lire.

À dire

[Contexte, 40 secondes]

Le Bénin a engagé la mise en place d'un système national de vidéoprotection, décidé en Conseils des Ministres de mars et juin 2026. Au-delà de la sécurité publique, un tel réseau ouvre la voie à la vidéo-verbalisation, c'est-à-dire constater automatiquement une infraction à partir des images. Cette chaîne compte six maillons, de la détection du véhicule jusqu'au procès-verbal, et tous dépendent du premier. Si un véhicule n'est pas détecté, aucun maillon en aval ne peut le rattraper.

[Problème, 40 secondes]

Or les détecteurs génériques échouent précisément dans les configurations les plus fréquentes de notre trafic : les véhicules éloignés, qui occupent peu de pixels, et les véhicules masqués dans les embouteillages. Ce n'est pas un défaut de programmation, c'est structurel : après les réductions de résolution successives du réseau, un véhicule de moins de 32 pixels de côté ne pèse plus que quelques valeurs, souvent insuffisantes pour déclencher une détection.

[Question de recherche, 20 secondes]

Notre question est donc la suivante : un fine-tuning de YOLOv8 sur du trafic urbain dense améliore-t-il la détection, et spécifiquement celle des véhicules de petite taille ?

[Données, 50 secondes]

Première difficulté, et nous l'annonçons nous-mêmes : il n'existe aucun jeu de données public de vision par ordinateur pour le Bénin. Nous avons donc travaillé sur BMD-45, un jeu de 45 000 images issues de caméras de vidéoprotection de Bengaluru, en Inde, publié sous licence Creative Commons. Nous l'avons choisi pour trois traits structurels partagés avec notre contexte : des caméras fixes, un trafic très dense, et une dominance des deux-roues. Nous en avons extrait, avec des graines fixées, un sous-ensemble reproductible de 3 000 images, soit 25 688 véhicules annotés, découpé en 2 400 images d'entraînement, 300 de validation et 300 de test.

[Protocole, 60 secondes]

Le cœur de notre travail est le protocole de comparaison, car comparer un modèle qui connaît 80 classes et un modèle qui en connaît 4 n'a rien d'évident. Nous avons pris trois précautions. Un : les 14 catégories du jeu source sont ramenées à quatre classes volontairement alignées sur COCO, et les étiquettes du test sont réindexées à la volée vers l'espace de classes du modèle évalué, de sorte que les deux modèles voient exactement les mêmes images et les mêmes boîtes de référence. Deux : le jeu de test est isolé, dérivé de façon déterministe, et n'intervient ni dans l'entraînement ni dans la sélection du modèle. Trois : nous ne rapportons pas seulement des moyennes globales, mais le rappel ventilé par taille d'objet, avec les aires ramenées à la résolution d'entrée du réseau. Sans cette normalisation, sur des images en haute définition, tous les véhicules tomberaient dans la catégorie « grand » et la mesure ne mesurerait plus rien. C'est d'ailleurs exactement ce qui s'est produit lors de notre premier essai.

[Résultats, 60 secondes]

Après 50 epochs sur un GPU T4, en 55 minutes, toutes les métriques progressent. La mAP@0,5 passe de 0,438 à 0,825, soit plus 88 pour cent. Mais le résultat qui répond à la question posée est ailleurs : le rappel sur les objets de moins de 32 pixels de côté passe de 0,122 à 0,694. Autrement dit, le modèle générique ne retrouvait qu'un petit véhicule sur huit, le modèle fine-tuné en retrouve plus des deux tiers. Et le gain est d'autant plus fort que l'objet est petit : plus 469 pour cent sur les petits, plus 136 sur les moyens, plus 20 sur les grands. Sur

les mêmes 300 images, cela représente 1 229 véhicules détectés en plus.

[Limites, 40 secondes]

Trois limites, que nous préférons énoncer nous-mêmes. Le domaine est indien et non béninois : nous démontrons que l'adaptation fonctionne, pas que le modèle est prêt pour Cotonou. L'échelle est modeste : 3 000 images sur 45 000, un modèle nano, 50 epochs. Et surtout, nos courbes montrent que l'entraînement n'avait pas convergé lorsque nous l'avons arrêté : nos chiffres sont donc un plancher, pas un plafond.

[Conclusion, 30 secondes]

La valeur de ce travail tient autant au protocole qu'aux chiffres. Nous livrons une chaîne reproductible et un protocole d'évaluation clef en main, prêts à être rejoués le jour où un jeu de données béninois existera. C'est d'ailleurs notre première perspective, avant même le passage à une architecture plus grande. Je vous remercie.

9.3 Les 20 questions de jury les plus dangereuses

Réponses en trois phrases maximum. Apprenez les cinq premières par cœur.

1. Votre jeu de données est indien, pas béninois. Que vaut votre conclusion ?

Elle vaut pour ce qu'elle affirme : un fine-tuning sur du trafic dense lève le verrou des petits objets. Nous ne prétendons à aucun moment avoir adapté le modèle au trafic béninois, et le rapprochement repose sur trois traits structurels explicités, caméras fixes, densité, dominance des deux-roues. La constitution d'un jeu béninois est notre première perspective, et notre protocole est prêt pour cela.

2. Un gain de 469 pour cent n'est-il pas un artefact d'un point de départ très bas ?

En partie oui, et nous le disons : un rappel qui part de 0,12 peut mécaniquement gagner beaucoup plus en pourcentage qu'un rappel qui part de 0,79. C'est pourquoi nous regardons aussi le gain en points : plus 57 pour les petits, plus 51 pour les moyens, plus 16 pour les grands, dans le même ordre. Le fait marquant reste qu'une catégorie inexploitable est devenue exploitable.

3. Comparer un modèle à 80 classes et un modèle à 4 classes, est-ce équitable ?

C'est le point que nous avons le plus travaillé. Les étiquettes du test sont réindexées à la volée vers l'espace du modèle évalué, et le modèle COCO est restreint aux quatre indices véhicules pour ne pas être pénalisé par des piétons comptés en faux positifs. Les deux modèles passent par la même fonction de mesure, appelée deux fois depuis le même script.

4. Pourquoi votre débit augmente-t-il alors que l'architecture est identique ?

Nous le rapportons comme une observation, pas comme un résultat. Deux explications plausibles : une suppression des non-maxima allégée par le passage de 80 à 4 classes, et la variabilité d'un environnement partagé comme Colab. Nous n'avons pas conçu d'expérience pour les départager, et le message défendable est simplement que le fine-tuning ne coûte rien en débit.

5. Avez-vous une contamination entre entraînement et test ?

Le risque existe et nous le documentons : BMD-45 est filmé par des caméras fixes, donc des images voisines peuvent se ressembler fortement. Nos splits proviennent de deux partitions distinctes du jeu source, et notre test est dérivé de la validation, donc sans image commune avec l'entraînement. Si une contamination subsiste, elle flatte les deux modèles de la même manière et n'invalidé pas la comparaison.

6. Votre entraînement avait-il convergé ?

Non, et c'est un résultat en soi. Les trois pertes de validation atteignent leur minimum aux epochs 46, 49 et 50, et l'arrêt anticipé, réglé à 15 epochs de patience, ne s'est jamais déclenché. La borne de 50 epochs vient de notre budget de calcul, donc nos chiffres sont un plancher.

7. Pourquoi ramener les aires à 640 pixels avant de les classer ?

Parce que les seuils COCO de 32 et 96 pixels supposent des images d'environ 640 pixels, alors que les nôtres font 1 920 pixels de large. Sans normalisation, presque tous les véhicules tomberaient dans la catégorie « grand » et la ventilation ne mesurerait plus rien. C'est exactement ce qui s'est produit lors de notre premier essai, et c'est ce qui nous a conduits à corriger le protocole.

8. Pourquoi avoir ignoré les trois-roues et les vélos ?

Parce qu'ils n'ont pas d'équivalent dans COCO. Les conserver reviendrait à demander au modèle de référence de détecter des catégories qu'il n'a jamais apprises, ce qui fausserait la comparaison en sa défaveur. L'exclusion protège donc le modèle de référence, pas le nôtre.

9. Pourquoi ne publiez-vous aucune métrique par classe ?

Parce que nos classes minoritaires sont trop peu représentées pour donner des métriques stables : 189 bus et 240 camions dans le jeu de test, contre 1 380 motos. Publier une valeur calculée sur si peu d'objets donnerait une fausse impression de précision. Nous rapportons en revanche les aires

sous courbe par classe issues de la validation, à titre indicatif.

10. Pourquoi la déclinaison nano et pas une plus grande ?

C'est un compromis assumé entre coût d'entraînement et représentativité, avec le budget de calcul dont nous disposons. C'est aussi la taille la plus plausible pour une inférence temps réel sur du matériel modeste, ce qui correspond au cas d'usage. Passer à la déclinaison *small* fait partie de nos perspectives.

11. Comment savez-vous que *mosaic* et *scale* sont responsables du gain sur les petits objets ?

Nous ne le savons pas, nous l'avancons comme l'explication la plus plausible. Isoler la contribution de chaque augmentation demanderait une étude d'ablation, c'est-à-dire réentraîner en désactivant un réglage à la fois. Nous ne l'avons pas menée, faute de temps machine, et nous l'écrivons dans la discussion.

12. Quelle est votre volumétrie exacte ?

3 000 images et 25 688 véhicules annotés, répartis en 2 400 images d'entraînement, 300 de validation et 300 de test. Par classe : 7 238 voitures, 14 497 motos, 1 625 bus, 2 328 camions. Cela fait 8,5 véhicules annotés par image en moyenne, ce qui est la signature d'un trafic dense.

13. Et la protection des données personnelles ?

Le jeu utilisé est public et diffusé sous licence Creative Commons BY 4.0, nous n'avons collecté aucune donnée. Les visages sont floutés à la source et nous avons flouté la seule plaque partiellement lisible de nos illustrations. Un déploiement réel supposerait un cadre conforme au code du numérique béninois et le contrôle de l'Autorité de protection des données personnelles.

14. Pourquoi deux seuils de confiance différents dans vos mesures ?

Parce qu'ils répondent à deux questions différentes. Le seuil très bas de 0,001 sert à tracer la courbe précision-rappel complète, dont la mAP est l'aire ; le seuil de 0,25 est un seuil d'exploitation réaliste, celui du rappel par taille et des visuels. Les deux sont appliqués identiquement aux deux modèles, ce que garantit le fait qu'ils soient écrits une seule fois dans le code.

15. Que se passerait-il si vous baissiez le seuil à 0,05 ?

Le rappel monterait et la précision baisserait, pour les deux modèles. La comparaison resterait valide puisque le seuil s'applique aux deux, mais les valeurs absolues changeraient. Nous avons choisi 0,25 parce que c'est un seuil d'exploitation courant et parce qu'il correspond au réglage par défaut de nos visuels.

16. Votre rappel par taille est agnostique à la classe. N'est-ce pas trop facile ?

C'est un choix explicite, cohérent avec la question posée, qui est « le véhicule a-t-il été vu » et non « a-t-il été correctement nommé ». Il s'applique aux deux modèles de la même manière, donc il n'avantage aucun des deux. La classification, elle, est évaluée séparément par la mAP et par la matrice de confusion.

17. Pourquoi votre rappel global vaut 0,714 alors que le total du rappel par taille donne 0,830 ?

Parce que ce sont deux mesures différentes. Le 0,714 est la moyenne des rappels par classe calculée par le validateur sur la courbe précision-rappel ; le 0,830 est un rappel agnostique à la classe, mesuré au seuil 0,25 sur l'ensemble des objets. Les deux sont exacts, ils ne répondent simplement pas à la même question.

18. Un seul entraînement, une seule graine. Quelle confiance accorder à vos chiffres ?

C'est notre limite la moins défendable, et nous la reconnaissons : nous ne rapportons aucun intervalle de confiance. Ce qui protège la comparaison, c'est que les deux modèles sont évalués sur exactement les mêmes données avec le même code, donc l'écart mesuré n'est pas attribuable à

une différence de protocole. Répéter l'expérience avec plusieurs graines est la première chose à faire pour consolider.

19. Que feriez-vous avec trois mois de plus ?

Dans l'ordre : constituer un jeu de données annoté de trafic béninois, car c'est le préalable à toute conclusion sur le domaine cible. Ensuite prolonger l'entraînement et passer aux déclinaisons plus grandes, puisque nos courbes montrent que nous n'avons pas convergé. Enfin, une étude d'ablation sur les augmentations et un test d'inférence par tuiles pour les très petits objets.

20. Votre modèle est-il déployable à Cotonou aujourd'hui ?

Non, et ce serait malhonnête de l'affirmer. Il est validé sur un domaine comparable, pas sur le domaine cible, et il n'a jamais vu une image béninoise. Ce qui est déployable dès aujourd'hui, c'est la chaîne complète et le protocole d'évaluation, qui permettraient de mesurer en quelques heures ce que vaut le modèle sur un premier lot d'images locales.

9.4 Journal de vérité : incohérences relevées dans le dépôt

Aucune de ces incohérences ne remet en cause les résultats. Les connaître évite d'être pris de court.

Où	In- co- hé- renc	Gravité
docs/STRUCTURE.md:123	an- nonc rest com non ver- sion alors qu'u ex- cep- tion de .git ver- sion le run yolo	documentation
docs/STRUCTURE.md:85-88	an- nonc les poid com non ver- sion- nés, alors que .git ver- sion yolo	documentation
article/figures/LISEZMOI.md:22-25	af- firm que rest n'est pas ver- sion il l'est	documentation
README.md:72-77 et docs/STRUCTURE.md:97-103	liste 5 scrip le dos- sier en cont	documentation

9.5 Le classement fait, non vérifiable, pas fait

Statut	Éléments
[fait et vérifié]	la volumétrie (recomptée), les 9 lignes du tableau comparatif, les hyperparamètres réellement utilisés (attestés par <code>args.yaml</code>), les courbes d'entraînement (recalculées depuis <code>results.csv</code>), le remappage des 14 classes, les seuils de taille et leur normalisation (reproduits à l'identique), le nombre de paramètres des deux modèles (mesuré), les aires sous courbe par classe
[fait mais non vérifiable dans le dépôt]	le matériel de mesure du débit, la durée d'entraînement hors du <code>results.csv</code> , la licence CC BY 4.0 de BMD-45, les chiffres de la littérature cités (facteur 2,5, 97,9 pour cent ougandais)
[pas fait]	répétition des expériences avec plusieurs graines, étude d'ablation sur les augmentations, entraînement jusqu'à convergence, test sur images béninoises, application de démonstration Streamlit, métriques par classe sur le jeu de test

À retenir

La posture qui gagne devant un jury technique n'est pas « nous avons tout réussi », c'est « voici ce que nous avons mesuré, voici comment, voici ce que cela ne prouve pas, et voici ce que nous ferons ensuite ». Vous avez les trois dans ce document.